

AIAA 2205: Introduction to AI

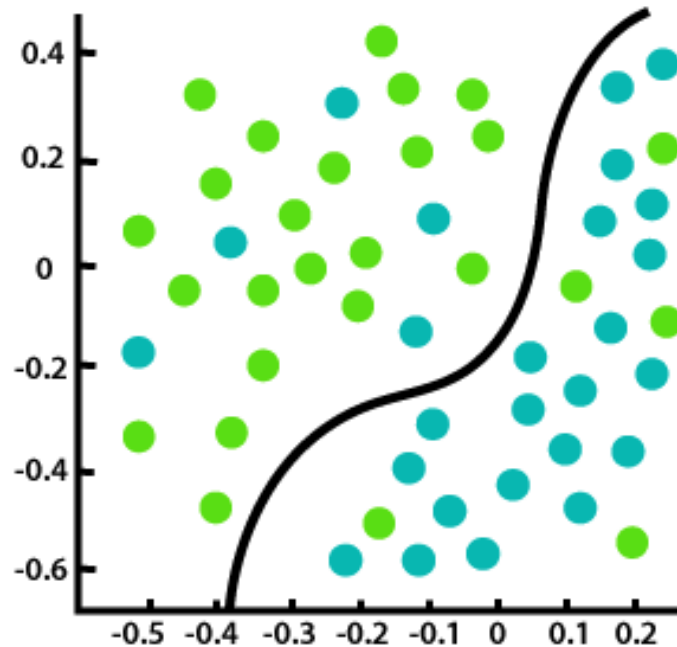
Enyan Dai

The Hong Kong University of Science and Technology (Guangzhou)

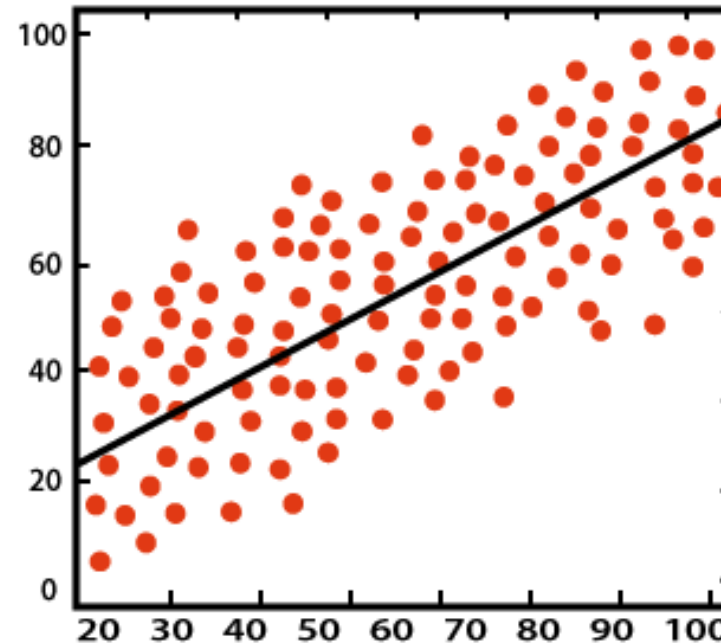
Recall of Last Lecture

Classification vs Regression

- **Classification:** used to predict/classify the **discrete** values such as Male or Female, True or False, Spam or not Spam, *etc.*
- **Regression:** used to predict the **continuous** values such as price, salary, *etc.*



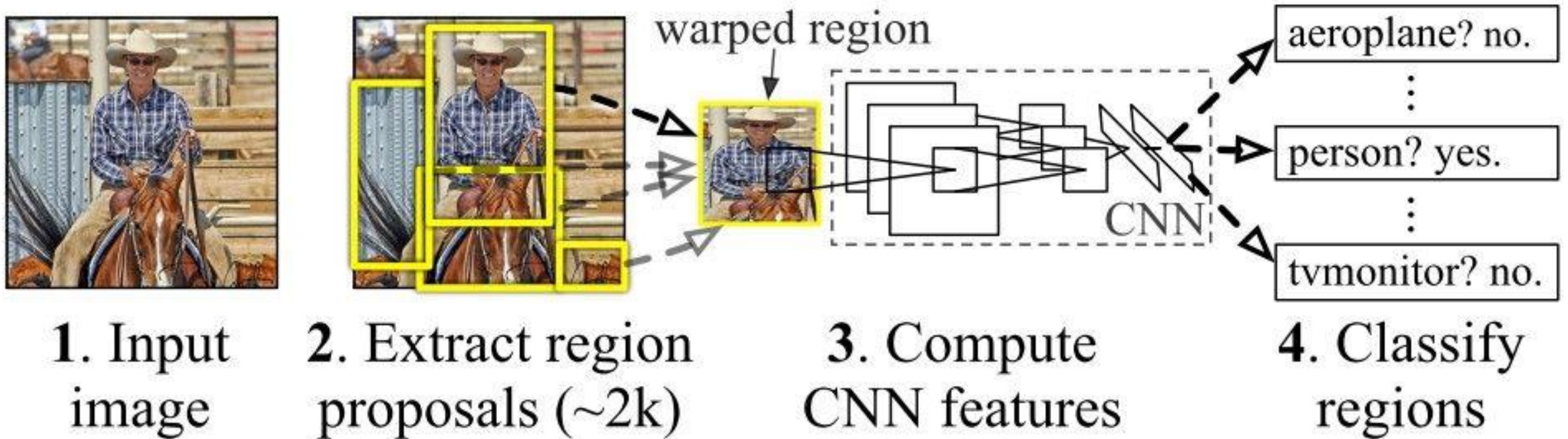
Classification



Regression

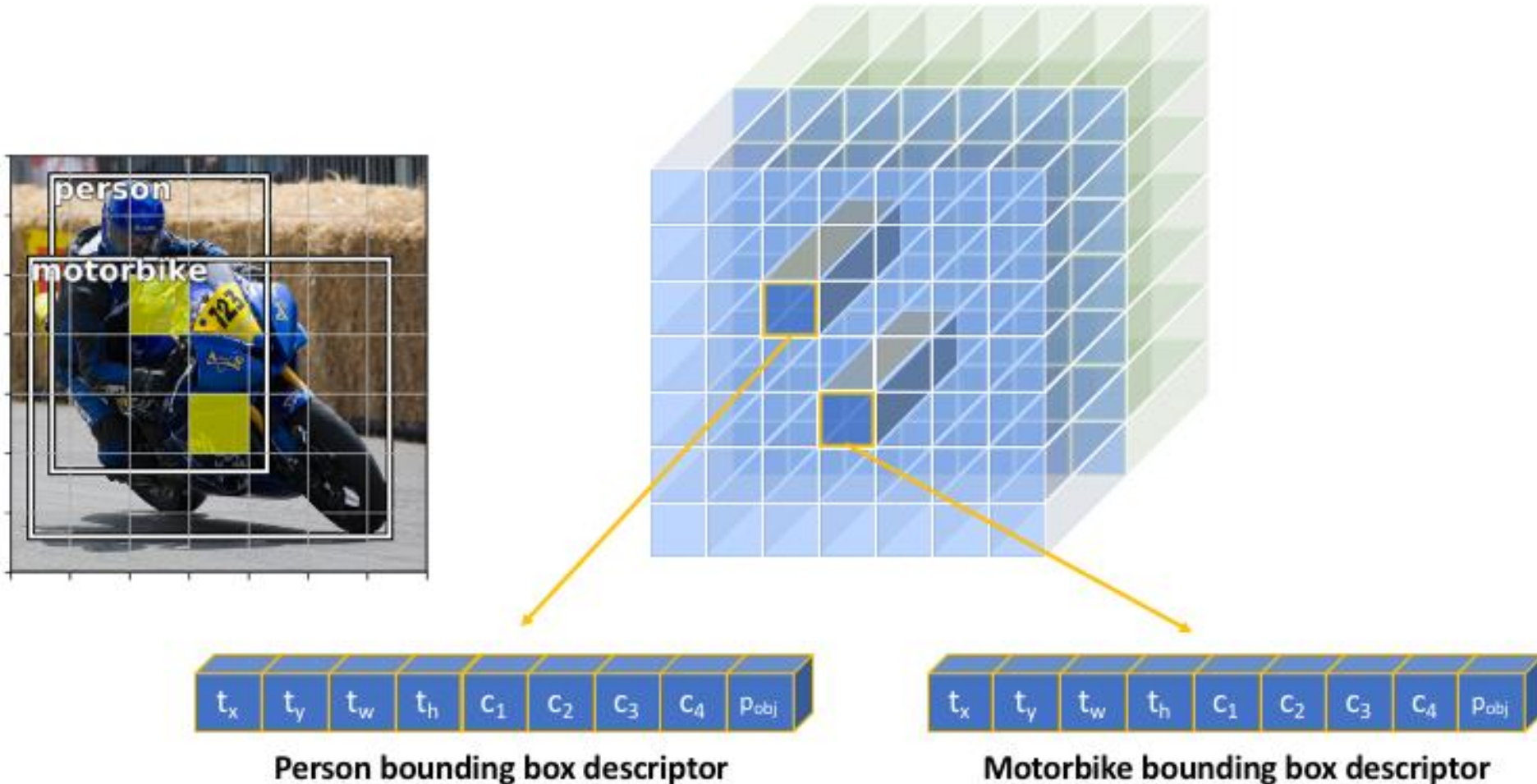
Object Detection

- Classical two-stage object detection is based on classification



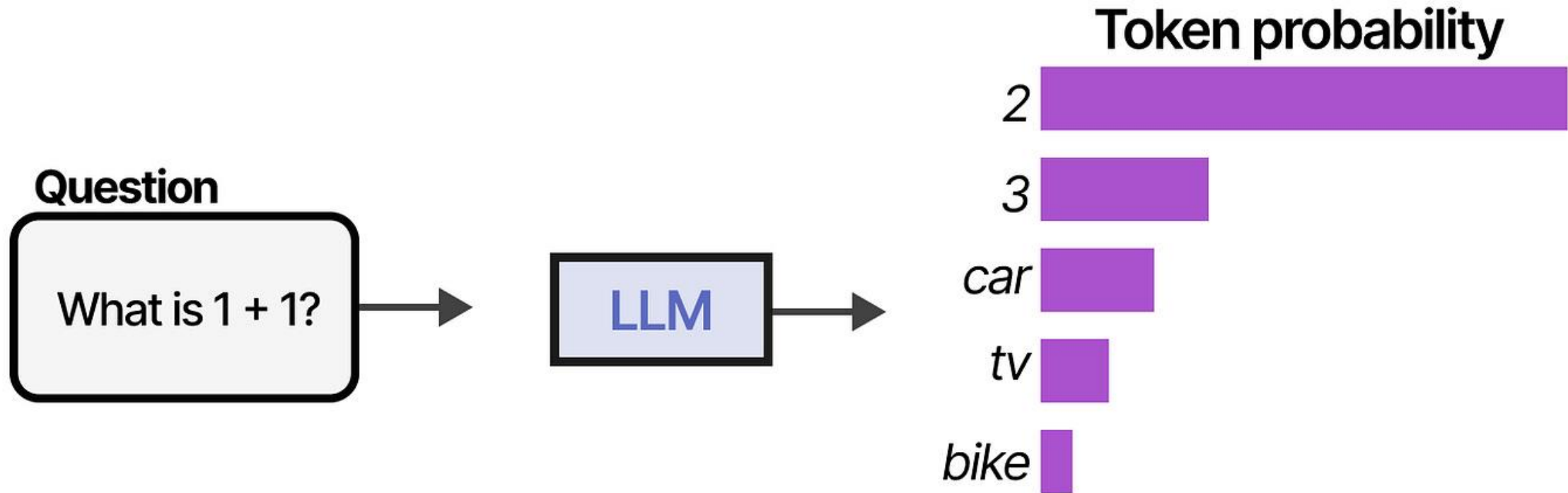
Object Detection

- One-stage object detection uses regressor to get bounding box



Large Language Model

- LLM is next token prediction, which is classification



Linear Regression

Given **training data**: $\mathcal{D} = \{(x_1, y_1), \dots, (x_N, y_N)\}$,
we aim to obtain a linear function:

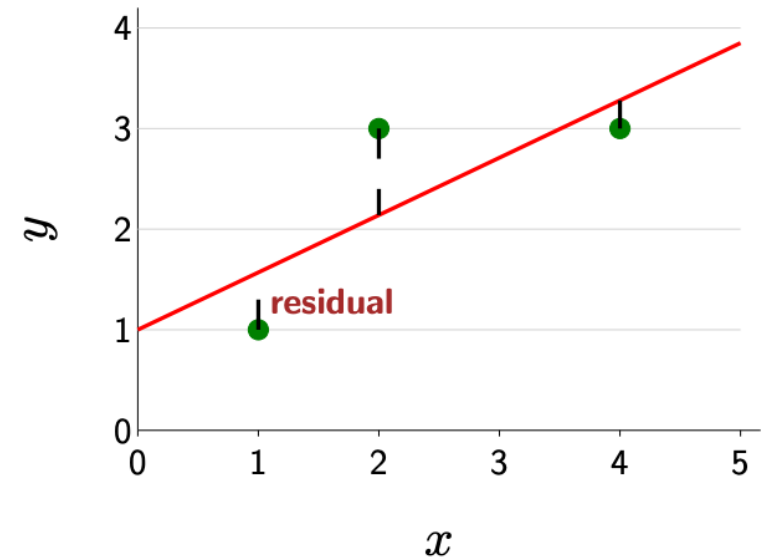
$$f(x) = \mathbf{w} \cdot \mathbf{x} + b,$$

where Mean Squared Error is minimized

	x	y
example	1	1
example	2	3
example	4	3

Mean Squared Error Loss:

$$\text{MSELoss}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (f(x_i) - y_i)^2$$



Solving w and b

A simplified formulation

$$\min_{w_b} \left\| w_a^T x_a - y \right\|^2$$

$$\begin{aligned} & \left\| w_a^T x_a - y \right\|^2 \\ &= (w_a^T x_a - y)(w_a^T x_a - y)^T \\ &= w_a^T x_a x_a^T w_a + y y^T - y w_a^T x_a - x_a^T w_a y \\ &= w_a^T x_a x_a^T w_a + y y^T - 2 y w_a^T x_a \end{aligned}$$

Let $\frac{\partial L}{\partial w_a} = 0$, then we have

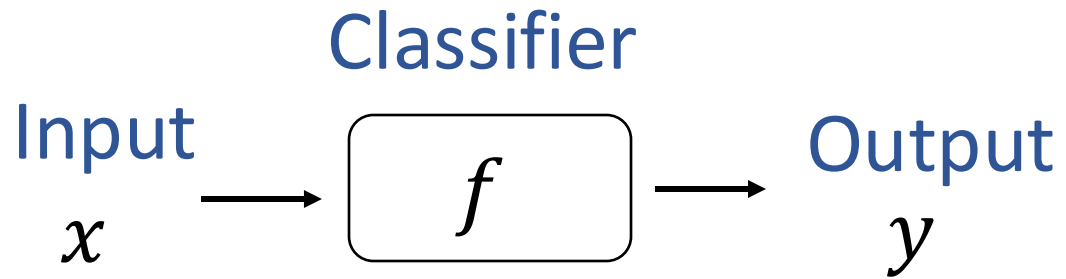
$$2 x_a x_a^T w_a - 2 x_a y = 0$$

A closed form solution

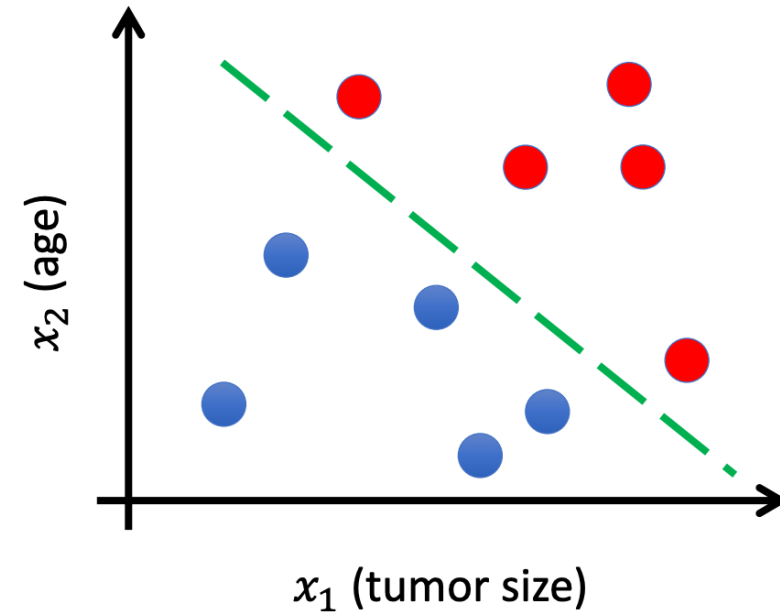
$$w_a = (x_a x_a^T)^{-1} x_a y$$

Logistic Regression

Logistic regression is used for **Binary Classification!**



- Binary: $y \in \{0, 1\}$
- Multiclass: $y \in \{1, \dots, K\}$



Example: **Malignant** vs. **Benign** Ocular Tumor

Logistic Regression

Sample	Probability to Estimate	Ground Truth
x_1	$P(y = 1 x_1) = p_1$	$P(y = 1 x_1) = 1$
x_2	$P(y = 1 x_2) = p_2$	$P(y = 1 x_1) = 0$

Adopt linear regressor $w \cdot x$ to estimate p ?

NO!!! Because $p \in [0,1]$

Solution: Adopt $w \cdot x$ to regress the logits of the event:

$$w \cdot x = \log\left(\frac{p}{1-p}\right) \in (-\infty, +\infty)$$

Logistic Regression

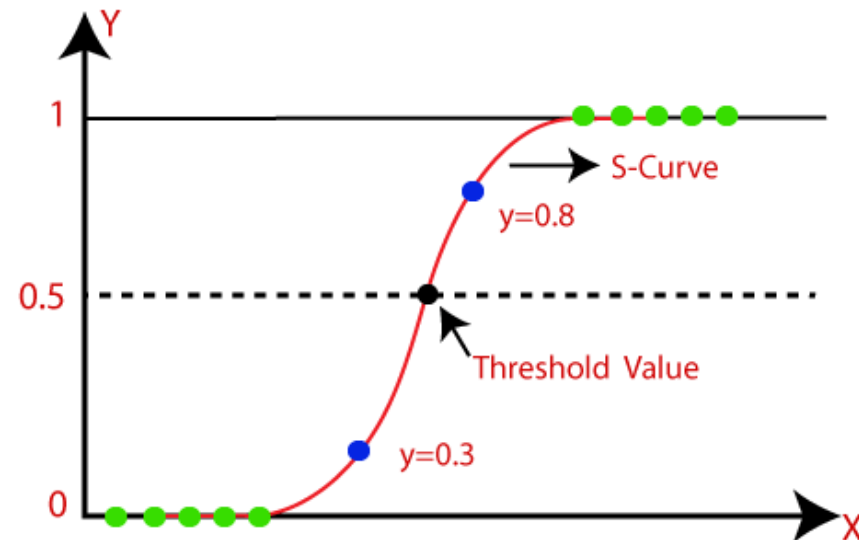
Estimate the Logit:

$$\mathbf{w} \cdot \mathbf{x} = \log\left(\frac{p}{1-p}\right) \Rightarrow f(\mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w} \cdot \mathbf{x})} = p$$

Sigmoid Function $\sigma(z) = \frac{1}{1 + \exp(-z)}$

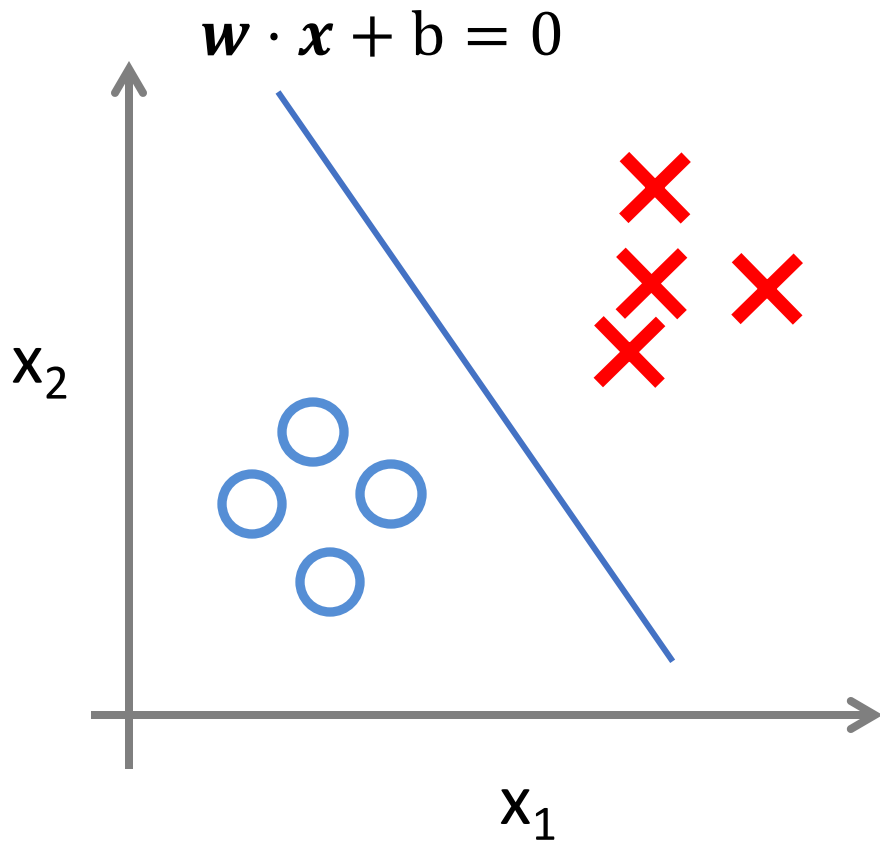
Decision Boundary: $\mathbf{w} \cdot \mathbf{x} = 0$

Optimization: Gradient Descent
with Cross Entropy Loss



Logistic Regression

Binary classification

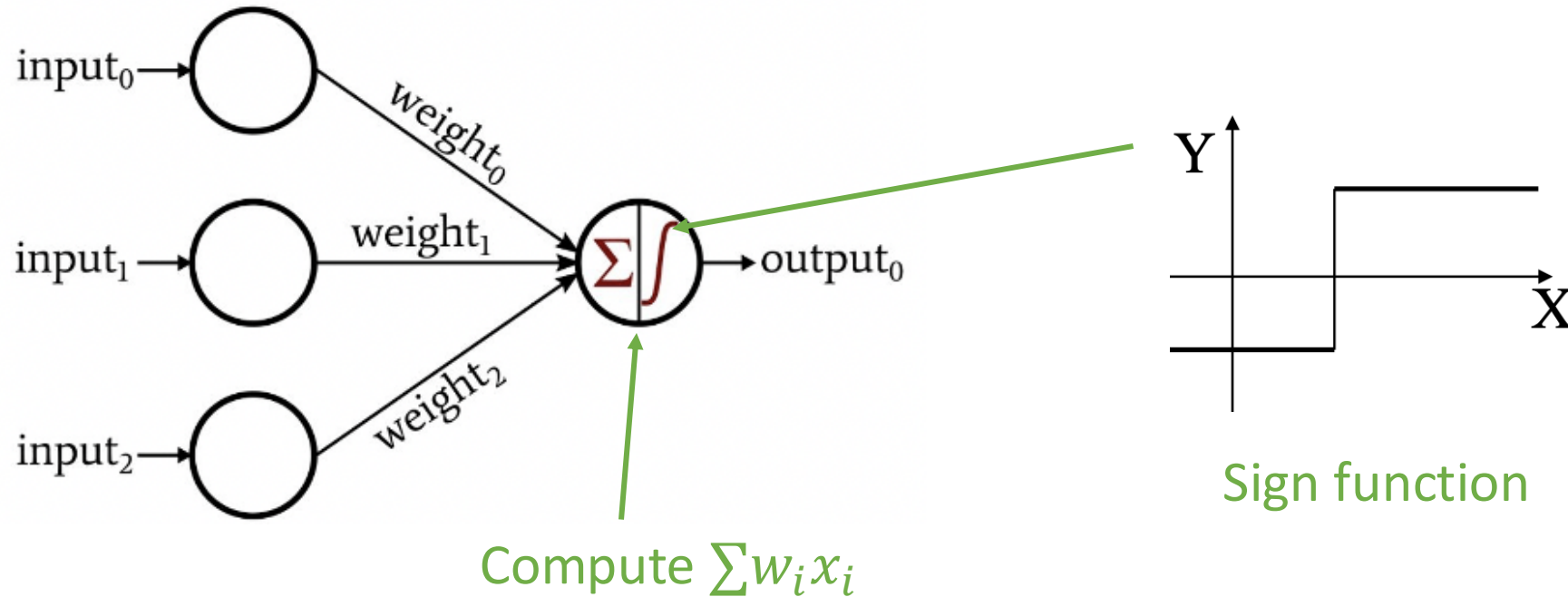


Predict $y = 1$, if $f(x) \geq 0.5$

Predict $y = 0$, if $f(x) < 0.5$

Perception

- Perceptron can be viewed as a logistics regression
- Equation: $y = \text{sign}(\sum w_i x_i)$

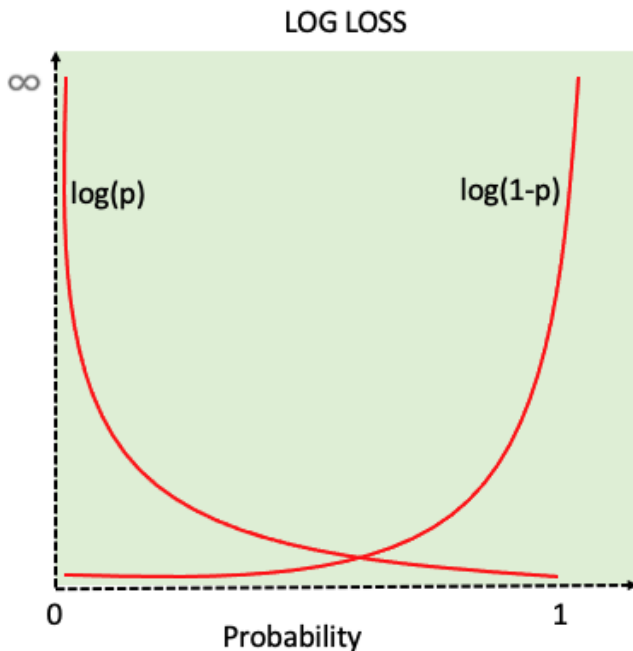


Cross Entropy Loss

$$H(f(x), y) = \begin{cases} -\log f(x) & \text{if } y = 1 \\ -\log(1 - f(x)) & \text{if } y = 0 \end{cases}$$



$$H(f(x), y) = -(y \log f(x) + (1 - y) \log(1 - f(x)))$$



Loss = 0:

- if $y = 1, f(x) = 1$
- If $y = 0, f(x) = 0$

Loss = $+\infty$:

- if $y = 1, f(x) = 0$
- If $y = 0, f(x) = 1$

Logistic Regression

Given **training data**: $\mathcal{D} = \{(x_1, y_1), \dots, (x_N, y_N)\}$,

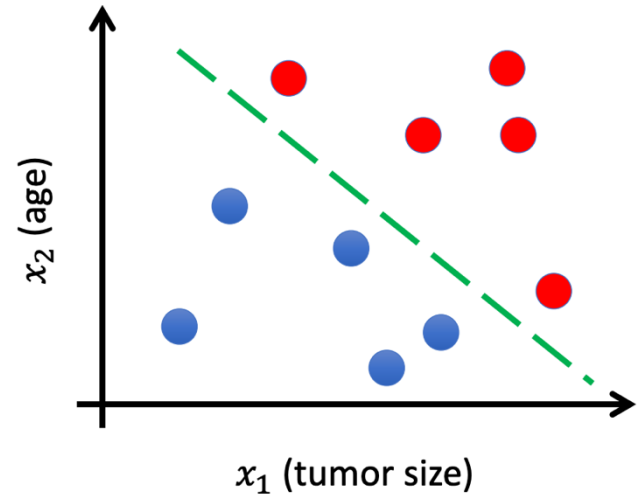
we aim to obtain a linear classifier:

$$f(x) = \text{sigmoid}(\mathbf{w} \cdot \mathbf{x} + b),$$

where cross entropy loss is minimized

Loss function:

$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^K H(f(x_i), y_i)$$



Example: **Malignant** vs. **Benign** Ocular Tumor

How to understand Cross Entropy Loss?

◆ 1. Self-Information: The Building Block

In information theory, the **self-information** (also called "surprisal") of an event with probability p is defined as:

$$I(p) = -\log(p)$$

Intuition:

- If something **unlikely** happens (low p), the self-information is **high** — it's more surprising.
- If something **very likely** happens (high p), the self-information is **low** — it's not surprising.

Example:

- If you are told something that had a 1% chance of happening:
 $I(0.01) = -\log(0.01) = 4.6 \text{ bits} \rightarrow \text{high information!}$
- If you are told something that had a 90% chance of happening:
 $I(0.9) = -\log(0.9) = 0.1 \text{ bits} \rightarrow \text{low information.}$

How to understand Cross Entropy Loss?

2. Entropy: Expected Self-Information

Definition:

Entropy $H(X)$ is the **average self-information** of a random variable X :

$$H(X) = \mathbb{E}_{x \sim p(x)}[I(x)] = - \sum_x p(x) \log p(x)$$

- Interpretation:

- Measures the "uncertainty" or "disorder" of X .
- Maximum when all outcomes are equally likely (e.g., fair coin).

Example:

- For a fair coin ($p = 0.5$):

$$H(X) = -0.5 \log_2(0.5) - 0.5 \log_2(0.5) = 1 \text{ bit.}$$

- For a biased coin ($p = 0.9$):

$$H(X) = -0.9 \log_2(0.9) - 0.1 \log_2(0.1) \approx 0.47 \text{ bits.}$$

How to understand Cross Entropy Loss?

3. Cross-Entropy: Expected Self-Information Under a Different Distribution

Definition:

Cross-entropy $H(p, q)$ measures the average self-information of events drawn from distribution p , but calculated using probabilities from another distribution q :

$$H(p, q) = \mathbb{E}_{x \sim p(x)}[-\log q(x)] = - \sum_x p(x) \log q(x)$$

• Key Idea:

- If q perfectly matches p (i.e., $q(x) = p(x)$), then $H(p, q) = H(p)$.
- Otherwise, $H(p, q) > H(p)$, with the gap equal to the **KL divergence** $D_{KL}(p||q)$.

Relation to Entropy and KL Divergence:

$$H(p, q) = H(p) + D_{KL}(p||q)$$

- $H(p)$: Intrinsic uncertainty of p .
- $D_{KL}(p||q)$: Penalty for approximating p with q .

How to understand Cross Entropy Loss?

4. Intuitive Example: Coin Flip Prediction

- **True Distribution (p):** Biased coin with $p(\text{heads}) = 0.6$, $p(\text{tails}) = 0.4$.

- Entropy:

$$H(p) = -0.6 \log_2(0.6) - 0.4 \log_2(0.4) \approx 0.97 \text{ bits.}$$

- **Model's Prediction (q):** Incorrectly assumes $q(\text{heads}) = 0.8$, $q(\text{tails}) = 0.2$.

- Cross-entropy:

$$H(p, q) = -0.6 \log_2(0.8) - 0.4 \log_2(0.2) \approx 0.6 \times 0.32 + 0.4 \times 2.32 \approx 1.06 \text{ bits.}$$

- KL divergence:

$$D_{KL}(p||q) = H(p, q) - H(p) \approx 1.06 - 0.97 = 0.09 \text{ bits.}$$

How to understand Cross Entropy Loss?

5. Cross-Entropy in Machine Learning

(1) Classification Tasks

- True Distribution (p): One-hot encoded labels (e.g., $[1, 0, 0]$ for class 0).
- Predicted Distribution (q): Softmax outputs (e.g., $[0.7, 0.2, 0.1]$).
- Cross-Entropy Loss:

$$H(p, q) = - \sum_i p_i \log q_i = - \log q_{\text{true class}}$$

- Example: If $p = [1, 0]$ and $q = [0.9, 0.1]$, then $H(p, q) = -\log(0.9) \approx 0.15$.

(2) Why Minimize Cross-Entropy?

- Minimizing $H(p, q)$ forces q to match p .
- Since $H(p)$ is fixed, minimizing $H(p, q)$ is equivalent to minimizing $D_{KL}(p||q)$.

AI深度研究员

AI深度研究员 



Gradient Descent: Optimize Differentiable Models

Closed form solution?

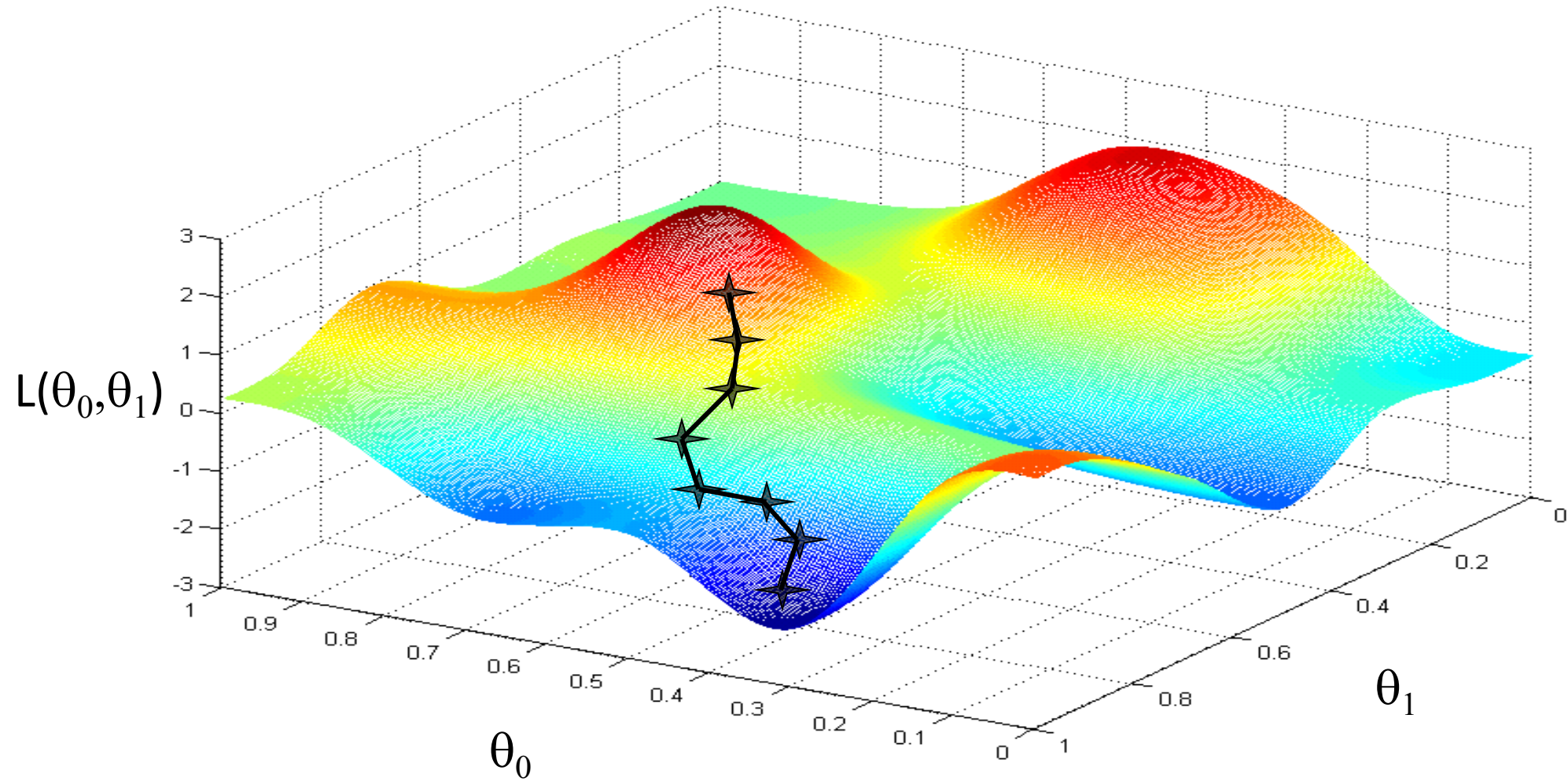
Closed form solution of linear regression:

$$\mathbf{w}_a = (\mathbf{x}_a \mathbf{x}_a^T)^{-1} \mathbf{x}_a \mathbf{y}$$

However, closed form solutions:

- Does not always exist in different machine learning models
- Could be very expensive for large-scale data

Intuition of Gradient of Descent



Gradient Descent

Optimized parameters $\mathbf{w}^*$:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} L(\mathbf{w})$$

- **Gradient** $\nabla_{\mathbf{w}} L(\mathbf{w})$ is the direction that increases the loss the most

Gradient Descent Algorithm

1. Initialize $\mathbf{w}^{(0)}$ by random values

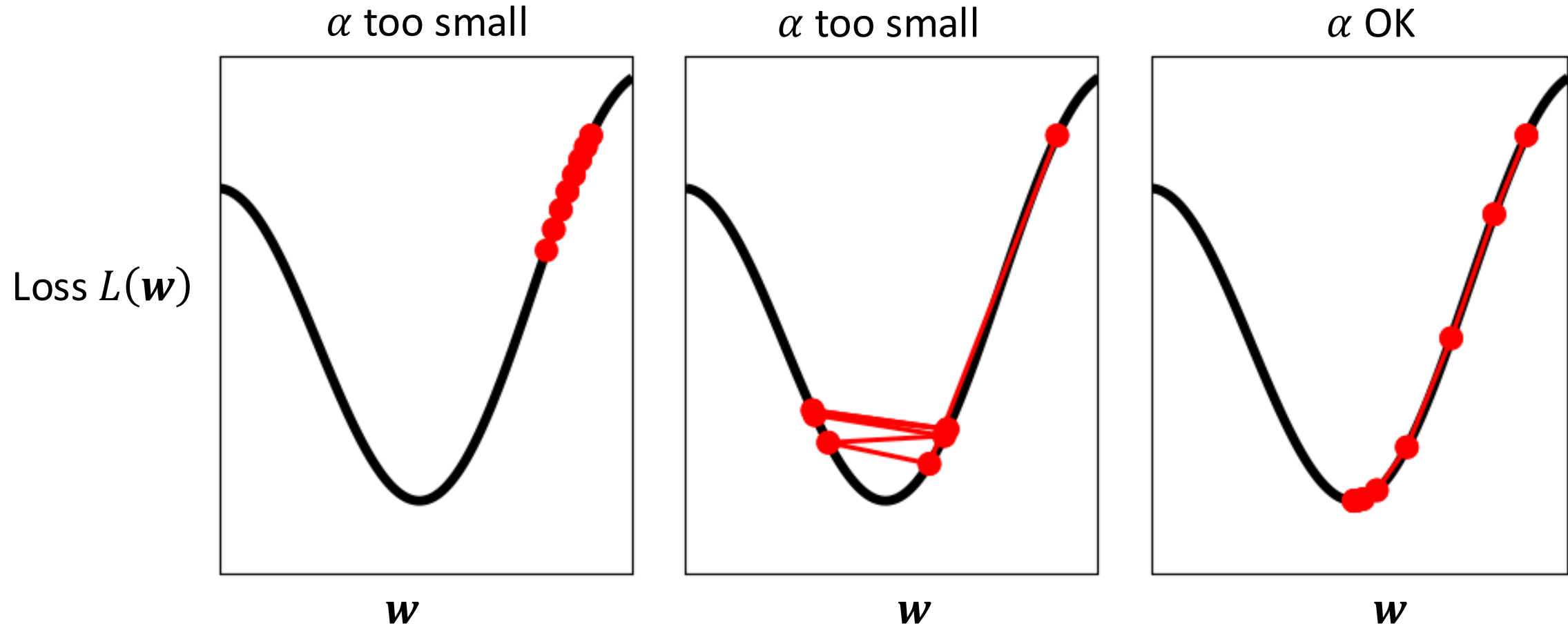
2. For $t = 1, \dots, T$:

$$\mathbf{w}^{(t)} \leftarrow \mathbf{w}^{(t-1)} - \alpha \nabla_{\mathbf{w}} L(\mathbf{w})$$

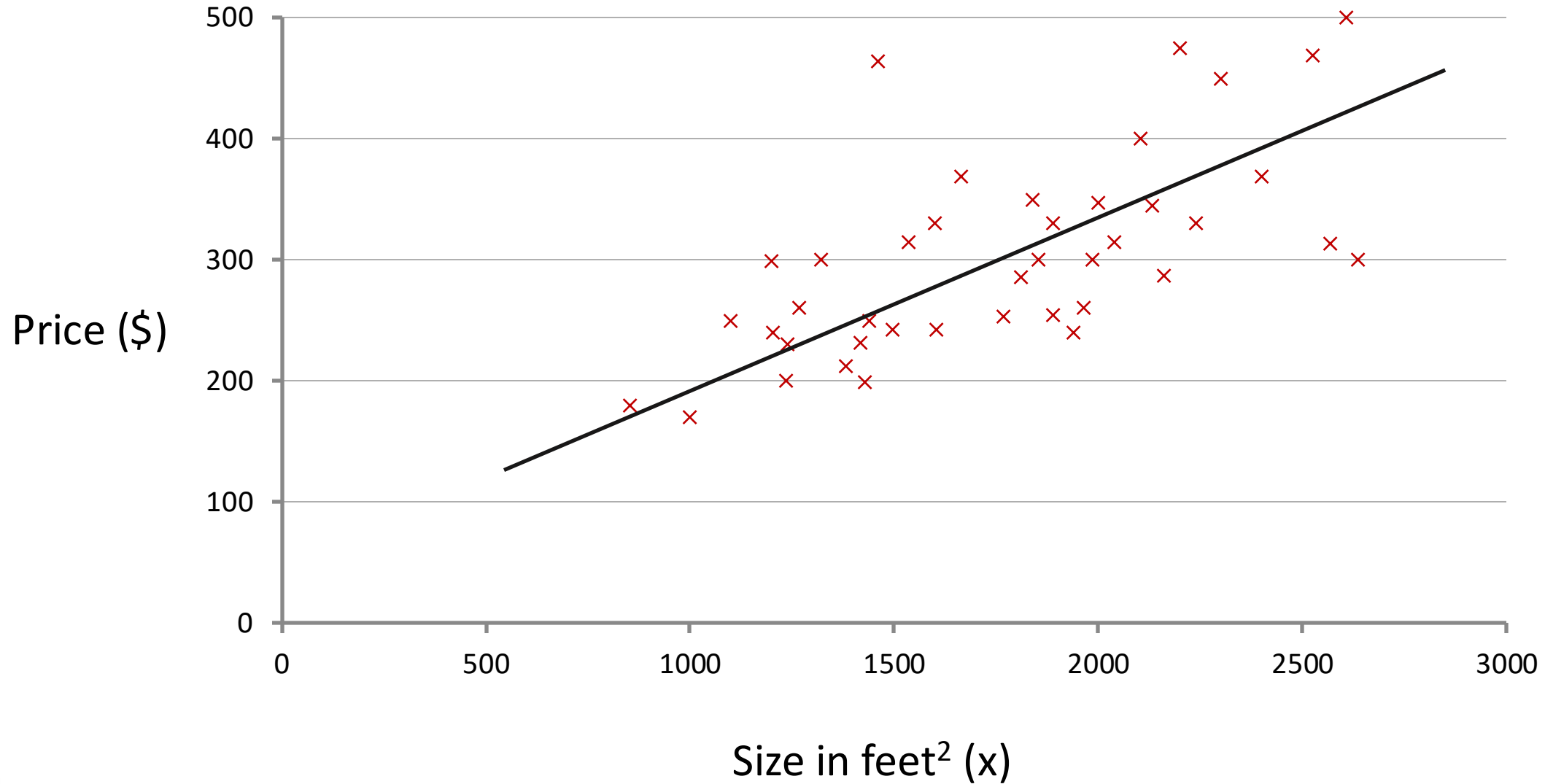
α : learning rate, step size in gradient descent

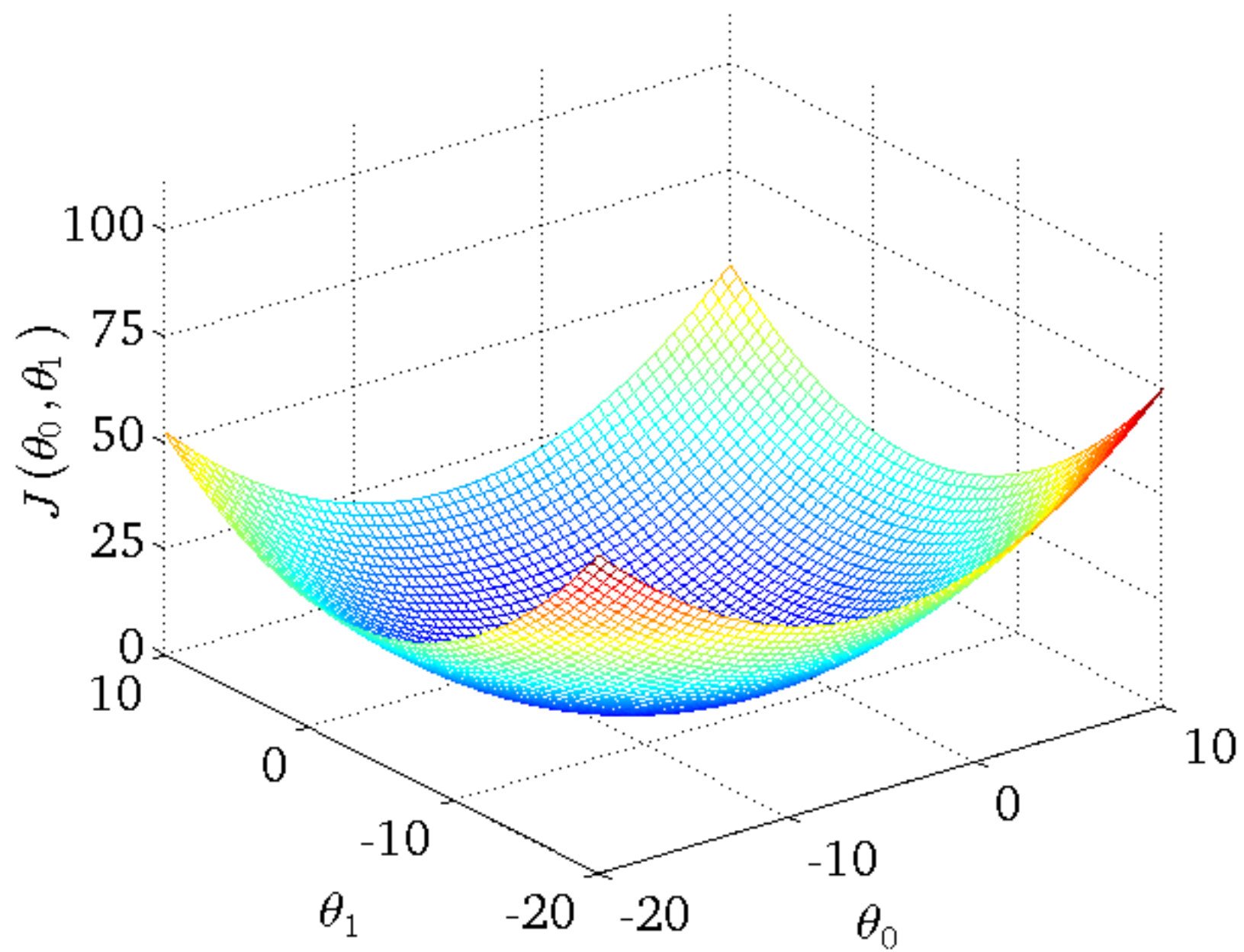
$\nabla_{\mathbf{w}} L(\mathbf{w})$: partial gradients of loss w.r.t parameters

Impacts of Learning Rates



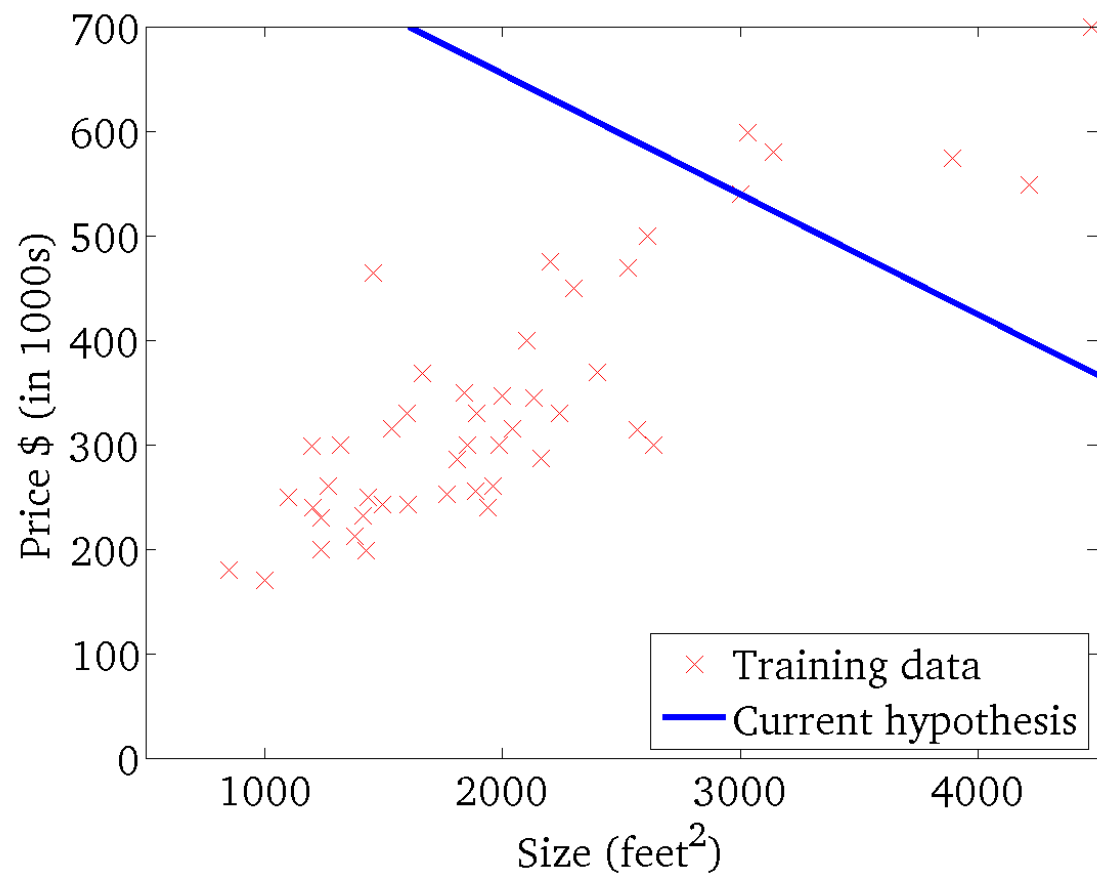
Example of Price Prediction





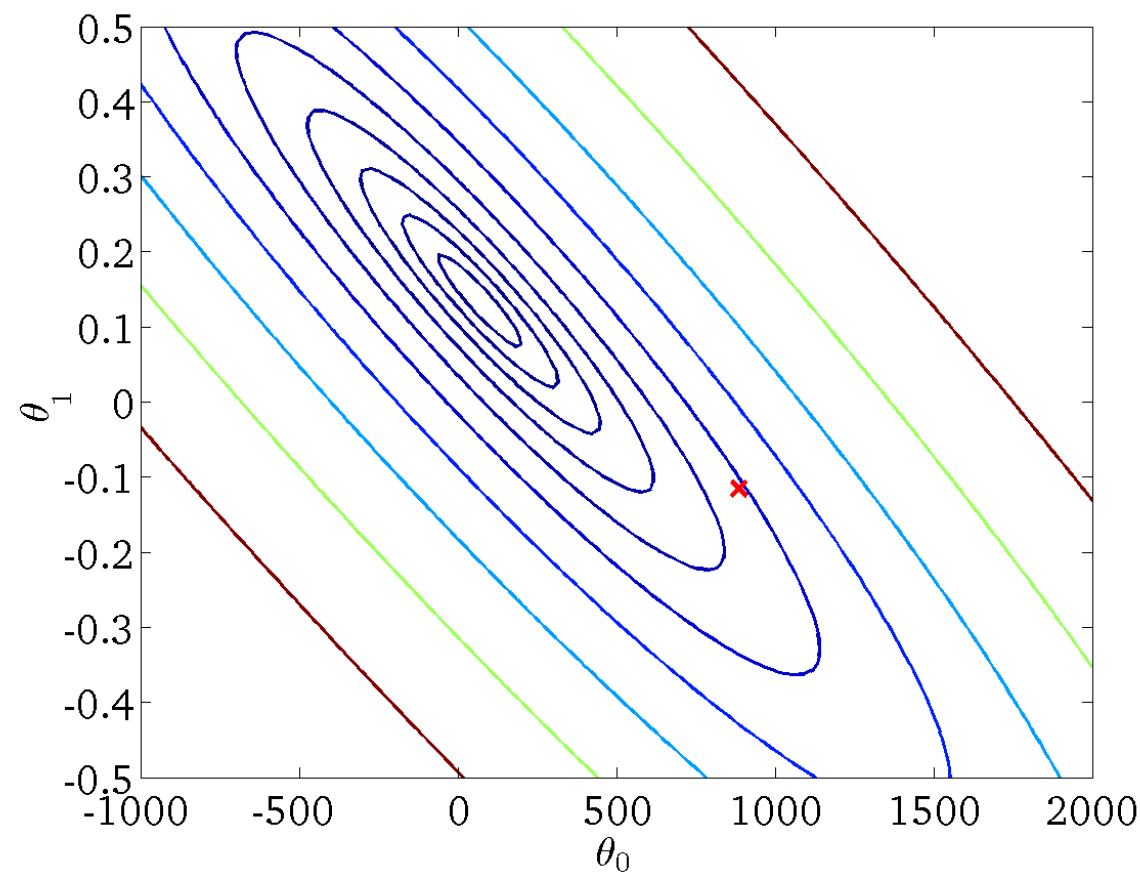
Regressor $f(\theta_0, \theta_1)$

(for fixed θ_0, θ_1 this is a function of x)



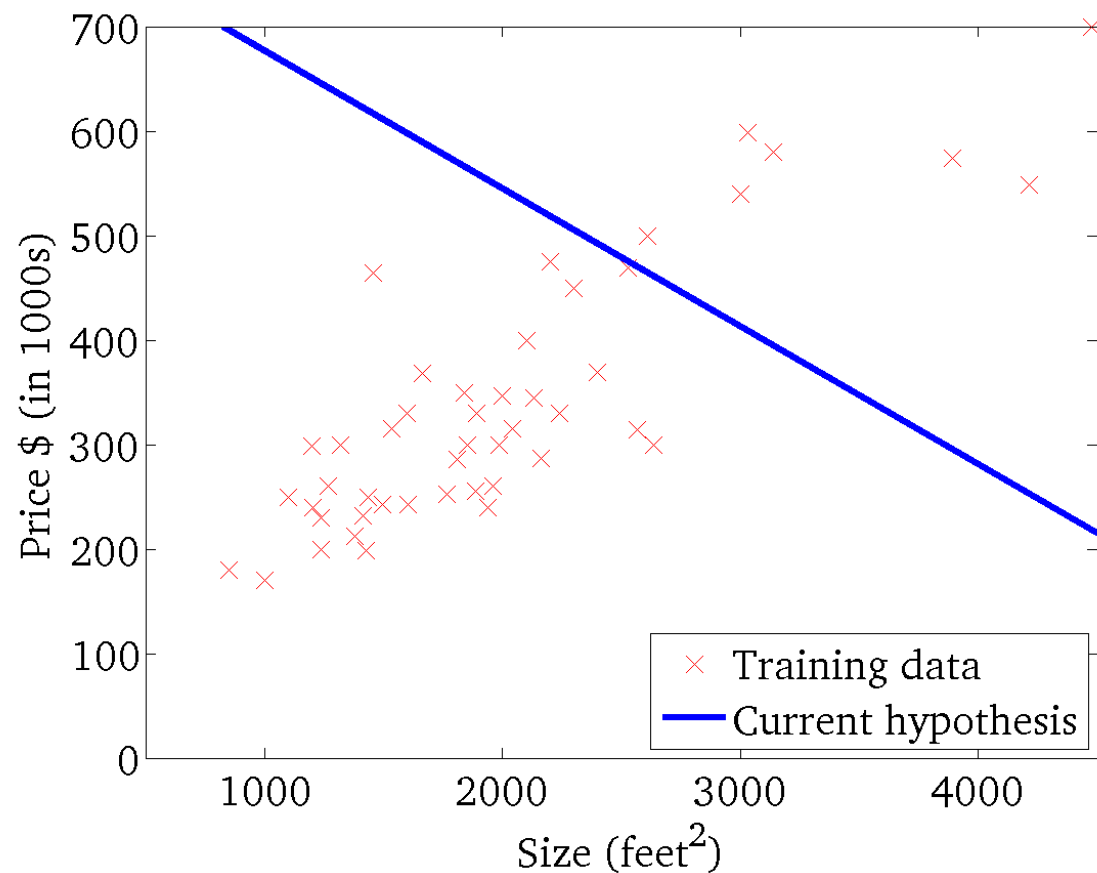
Loss $L(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



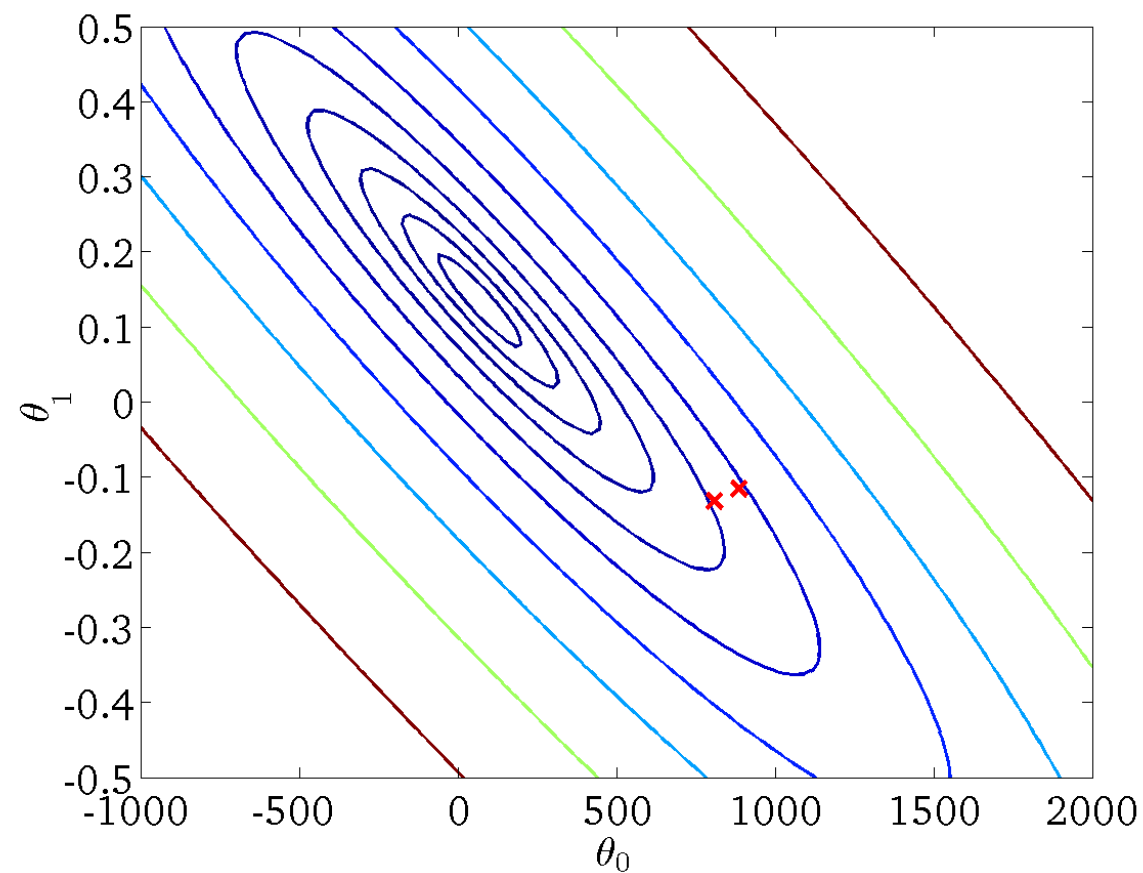
Regressor $f(\theta_0, \theta_1)$

(for fixed θ_0, θ_1 this is a function of x)



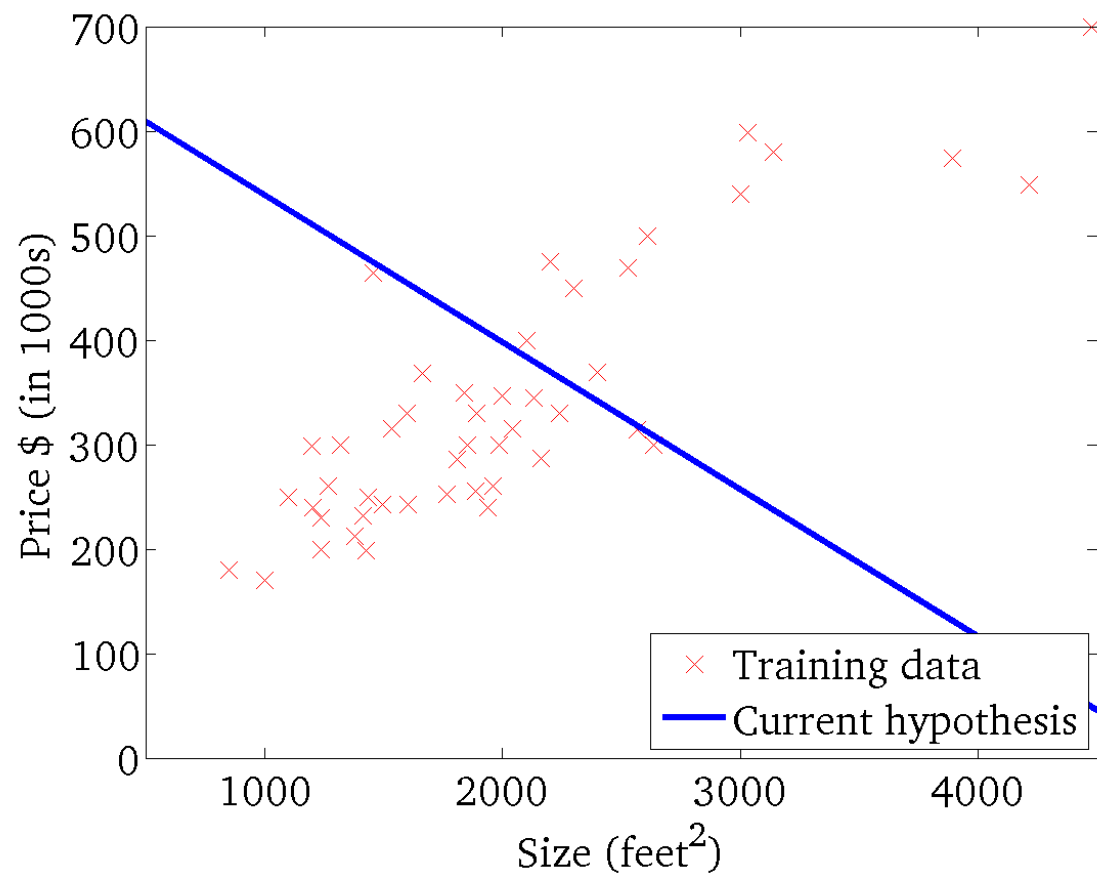
Loss $L(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



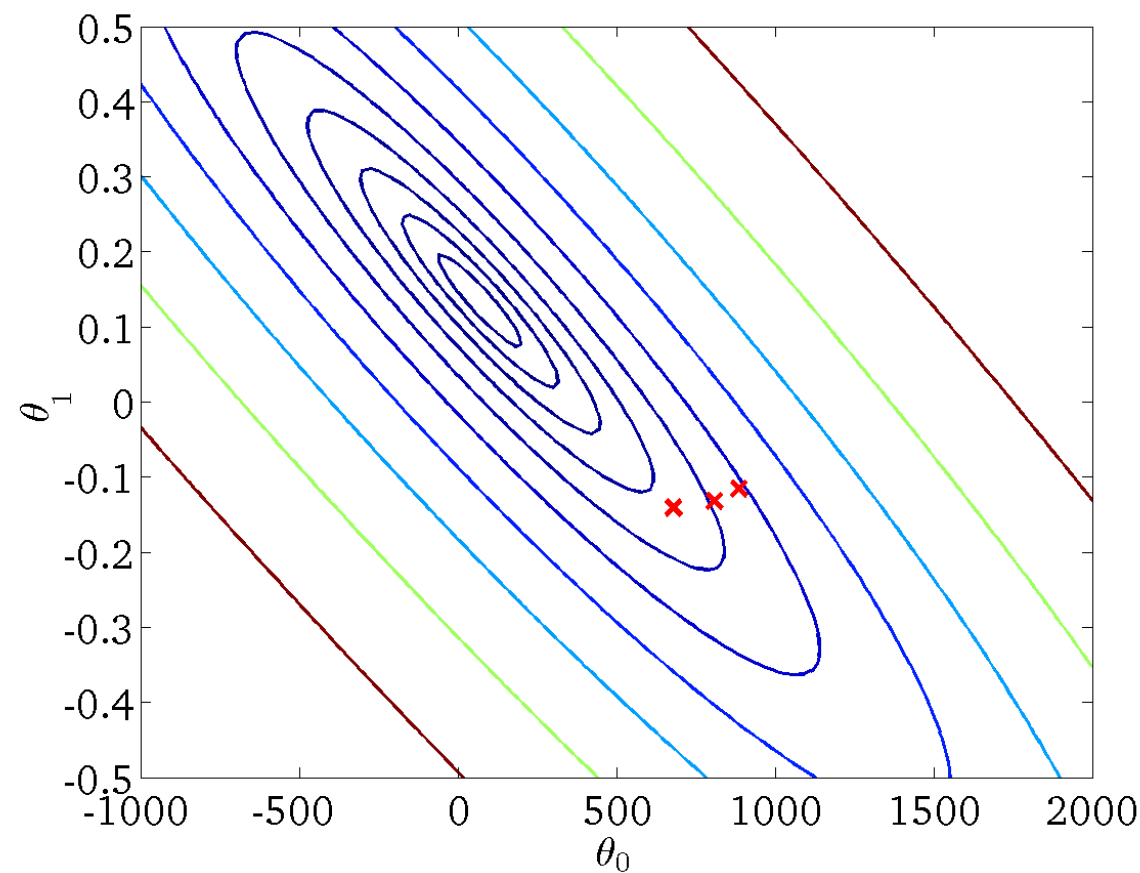
Regressor $f(\theta_0, \theta_1)$

(for fixed θ_0, θ_1 this is a function of x)



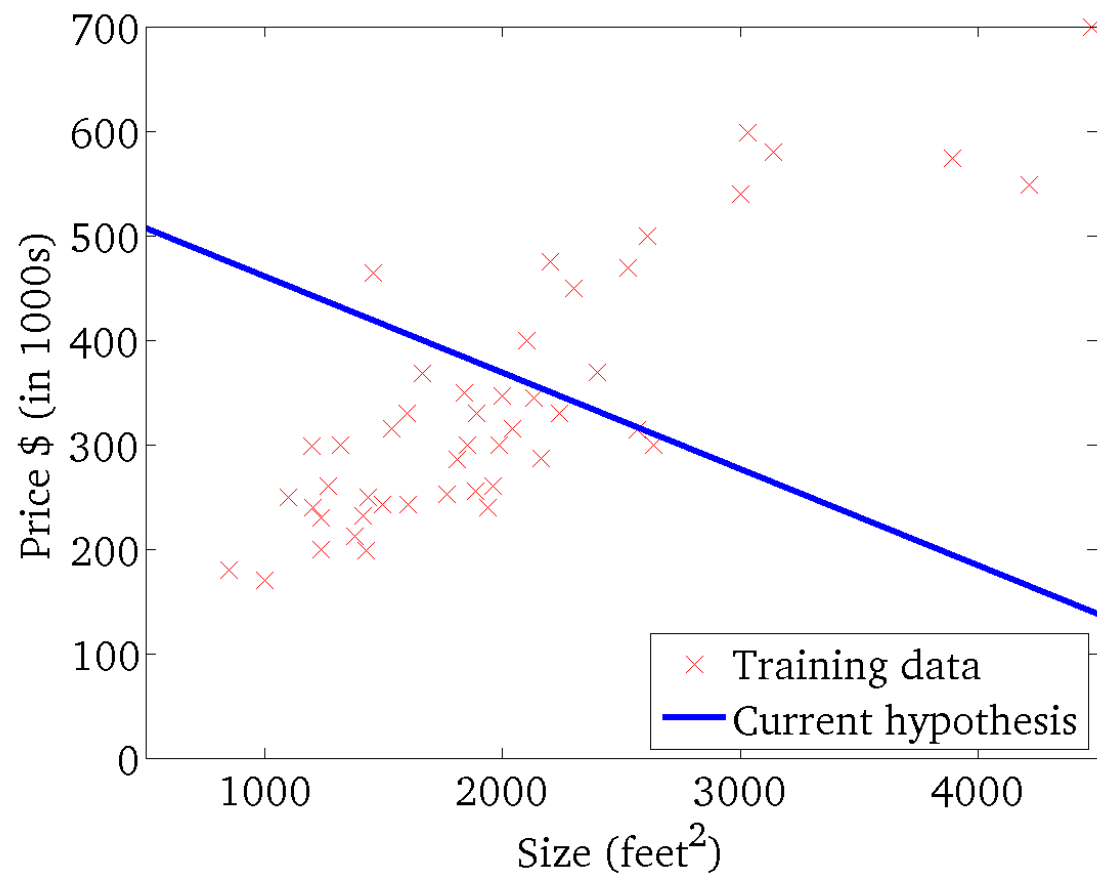
Loss $L(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



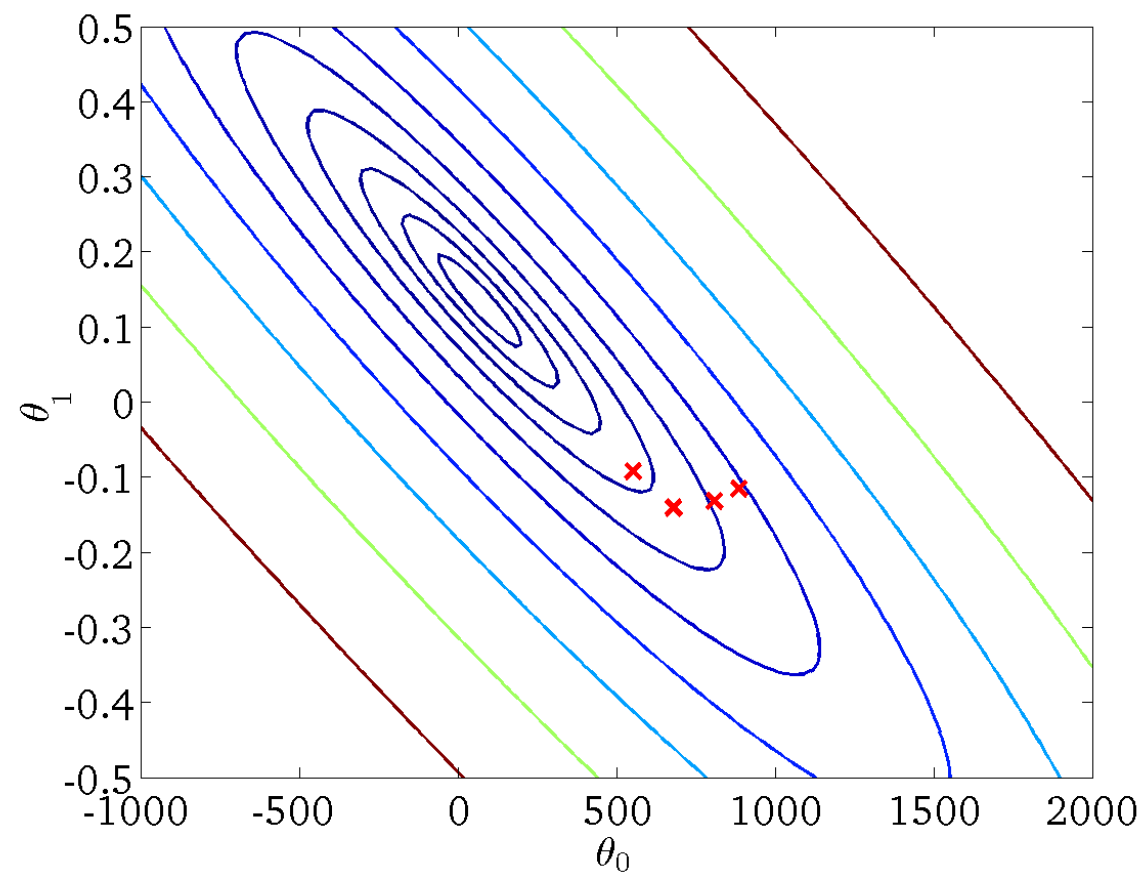
Regressor $f(\theta_0, \theta_1)$

(for fixed θ_0, θ_1 this is a function of x)



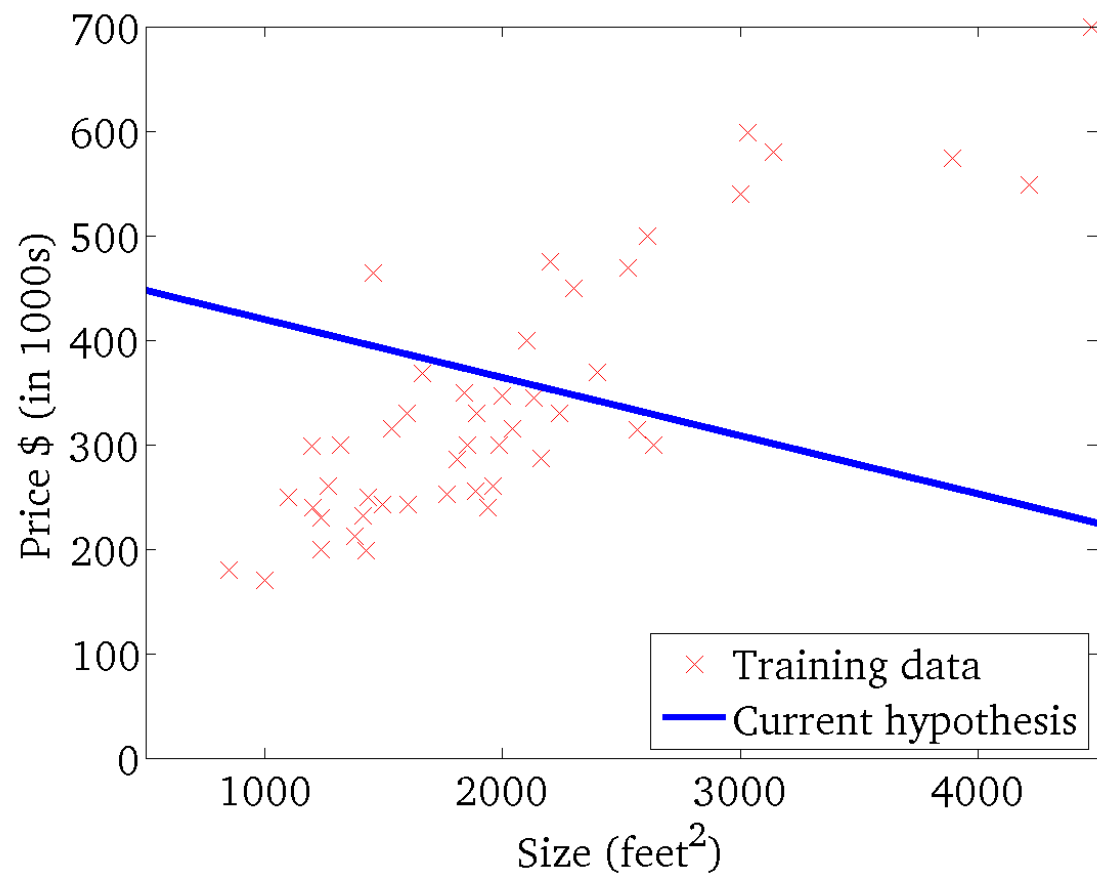
Loss $L(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



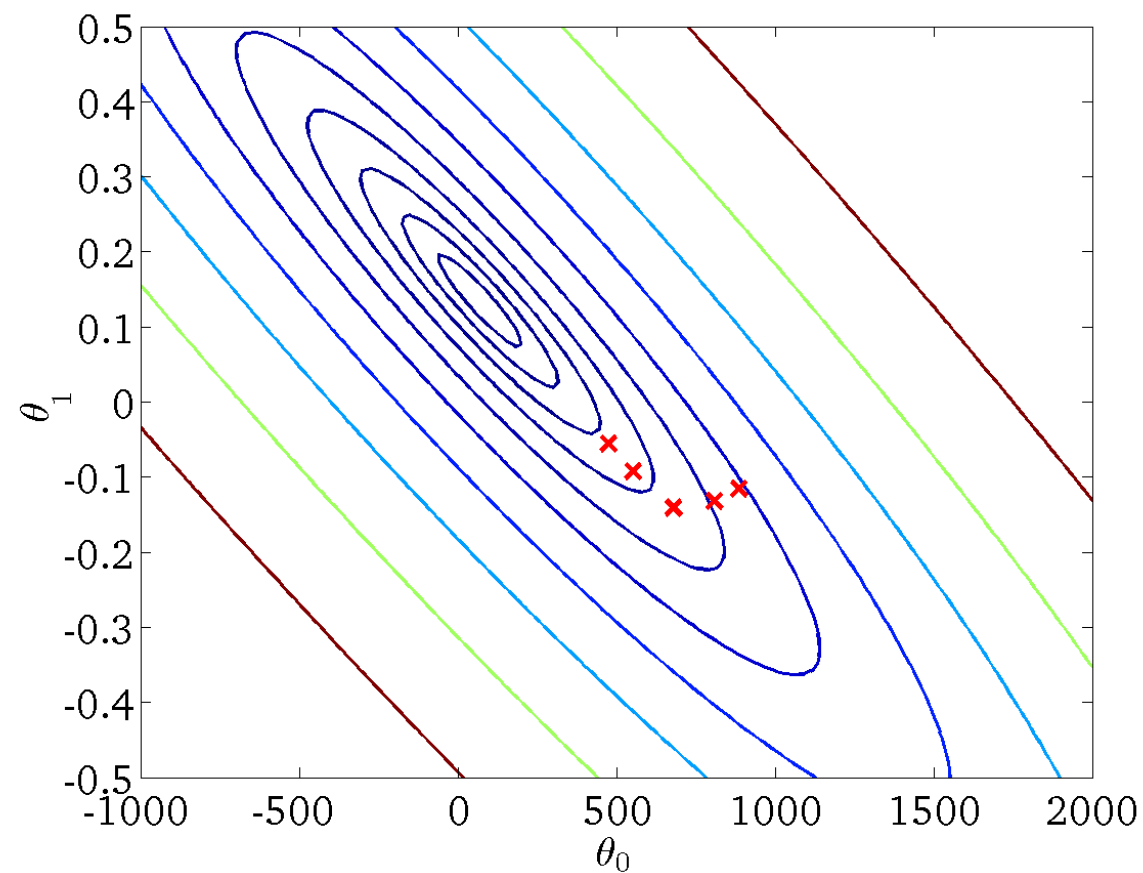
Regressor $f(\theta_0, \theta_1)$

(for fixed θ_0, θ_1 this is a function of x)



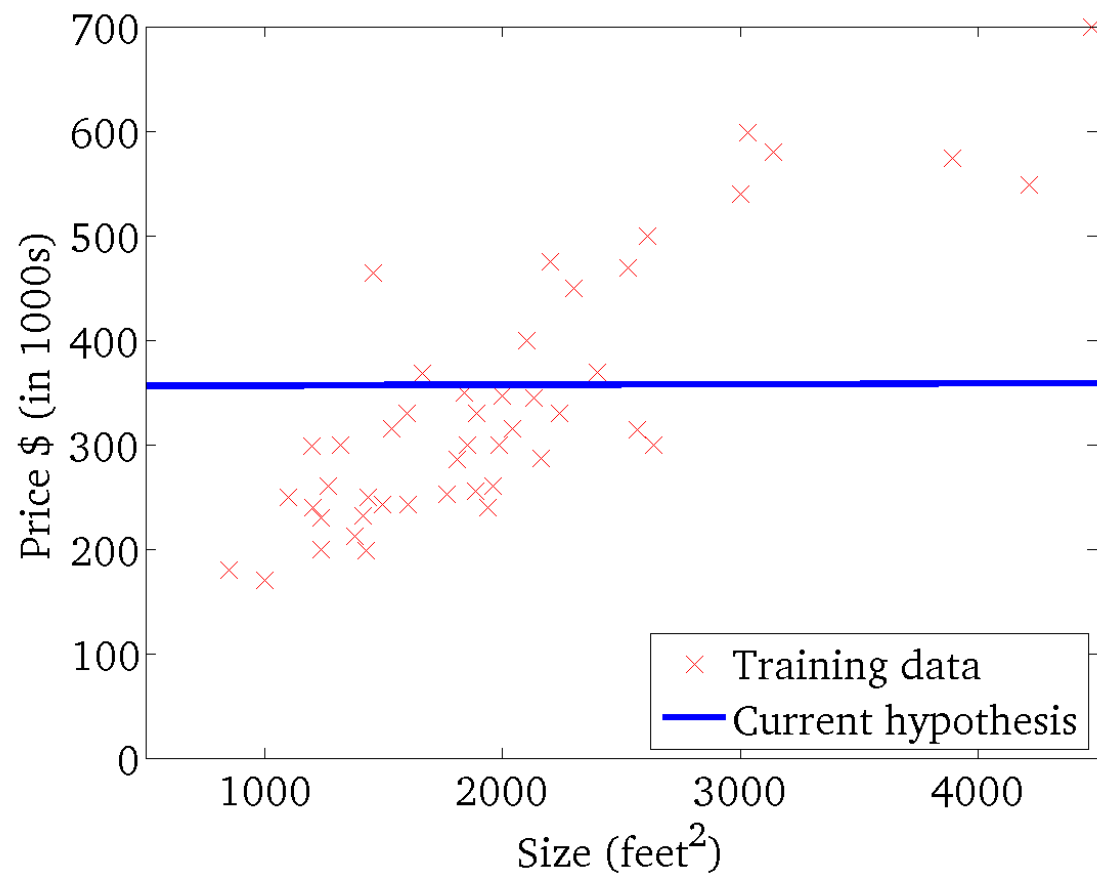
Loss $L(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



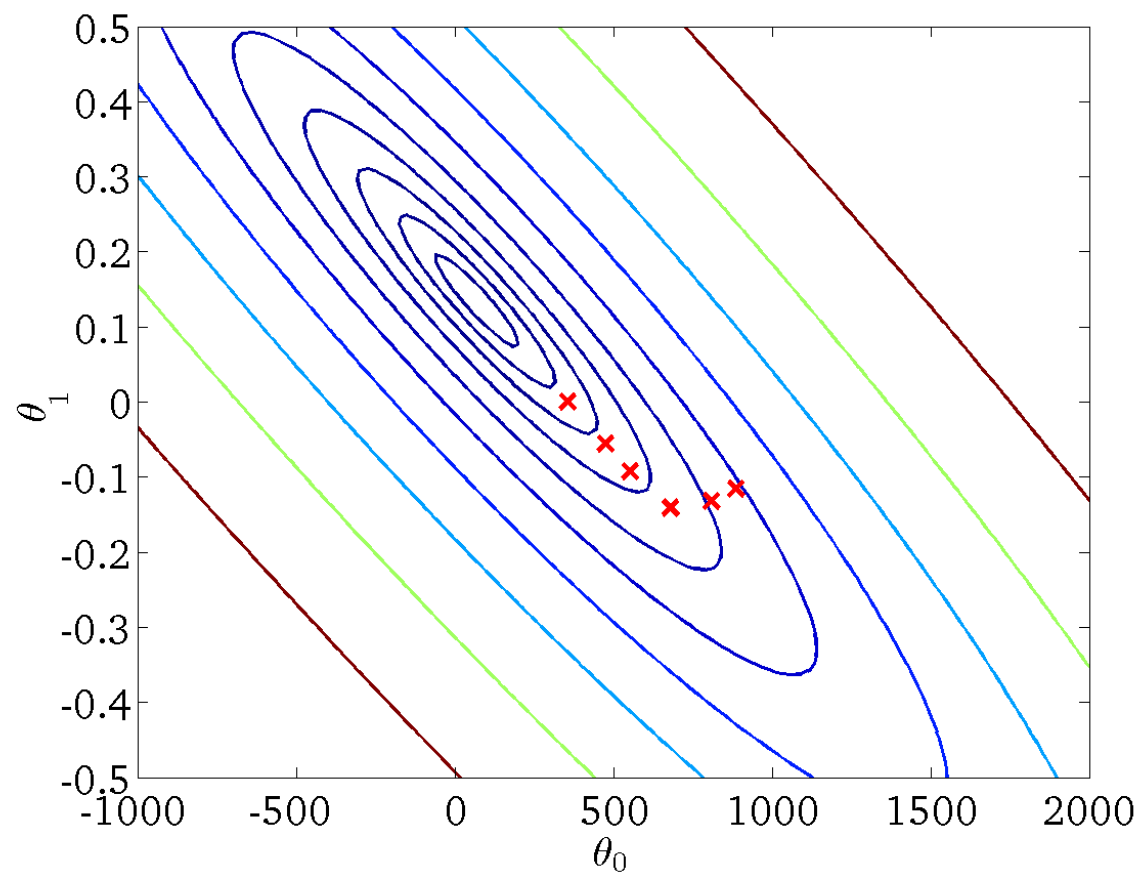
Regressor $f(\theta_0, \theta_1)$

(for fixed θ_0, θ_1 this is a function of x)



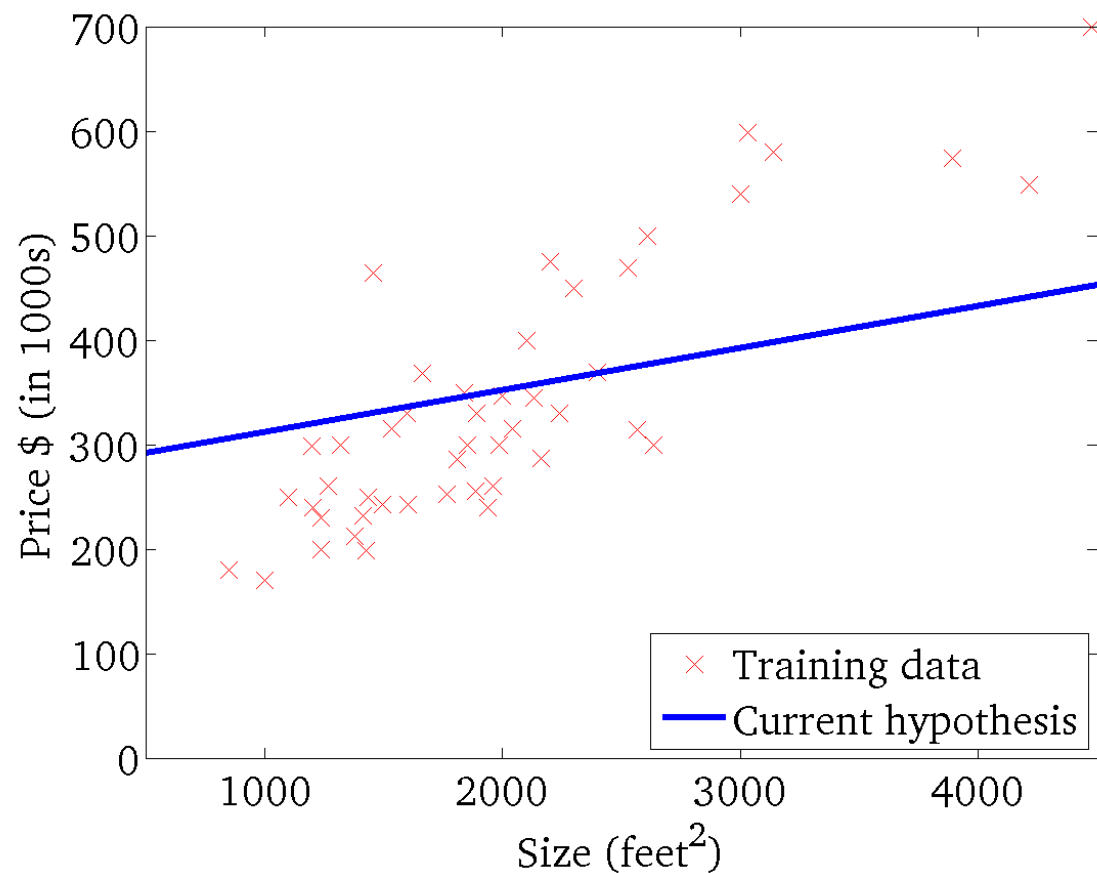
Loss $L(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



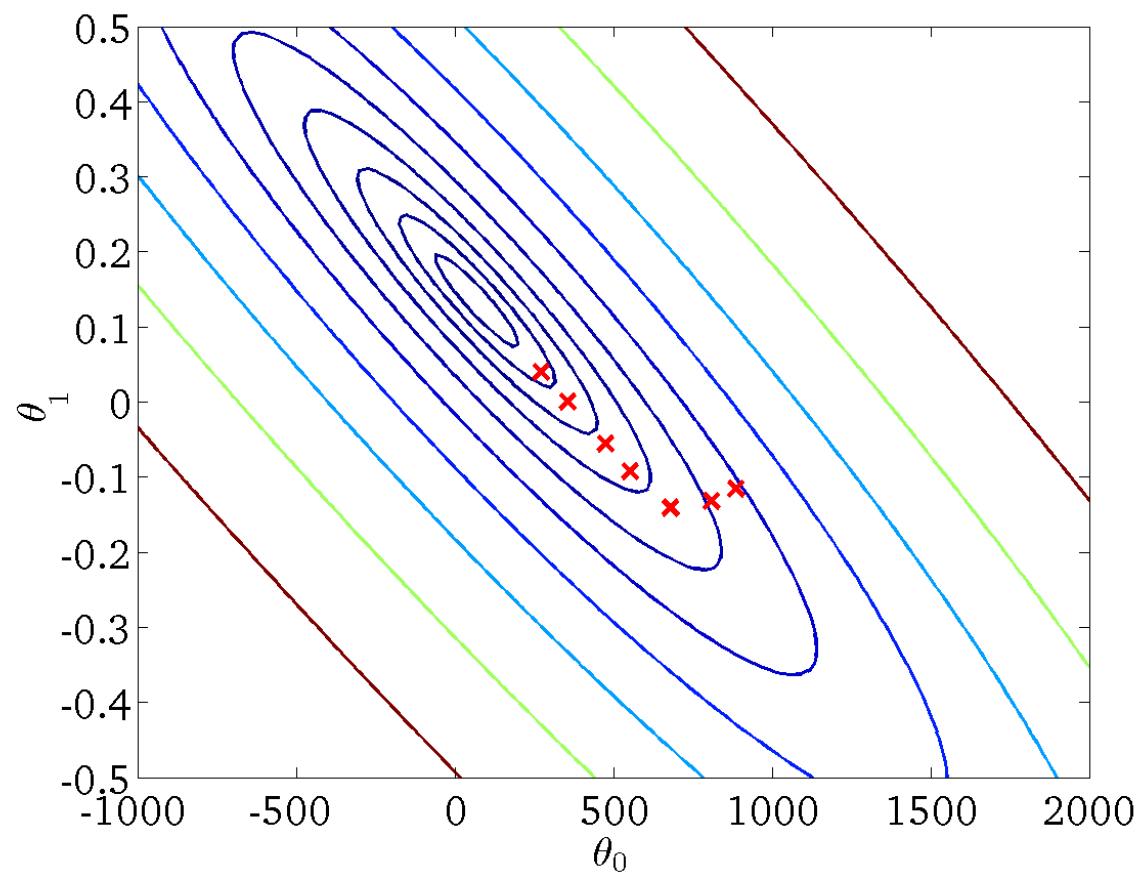
Regressor $f(\theta_0, \theta_1)$

(for fixed θ_0, θ_1 this is a function of x)



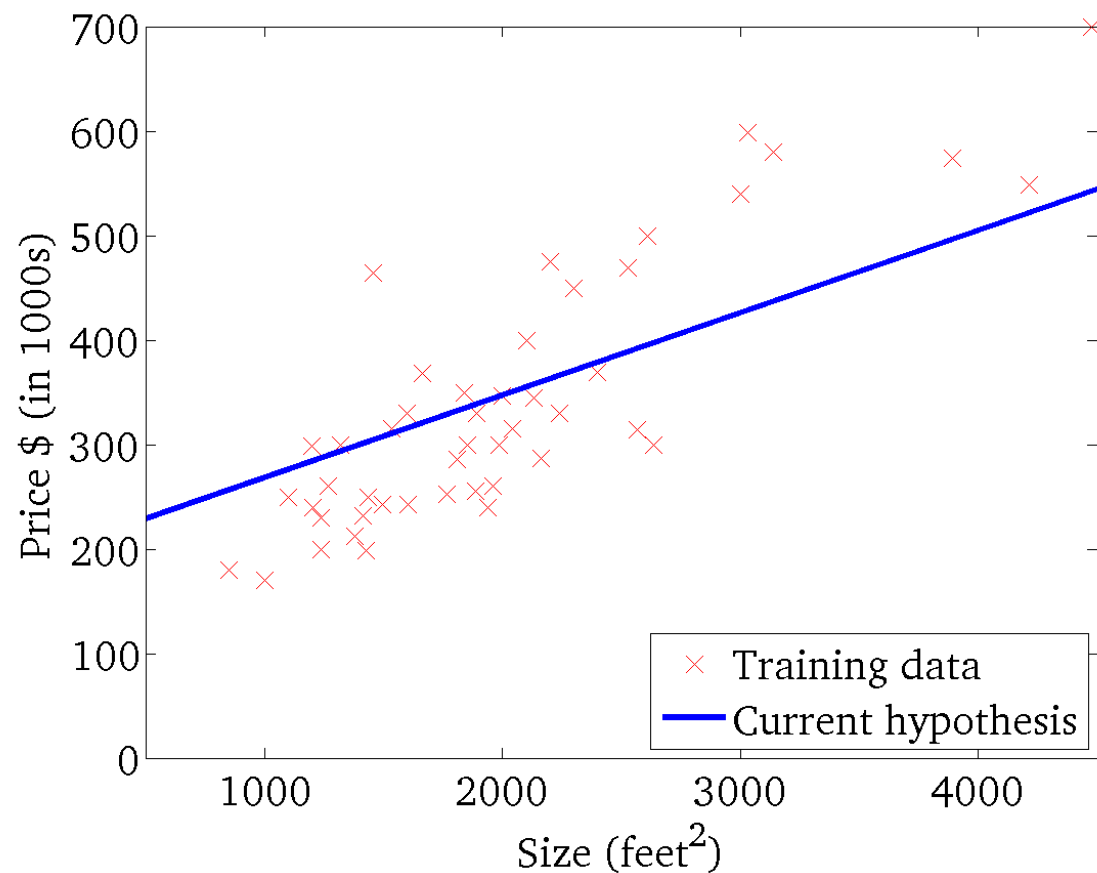
Loss $L(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



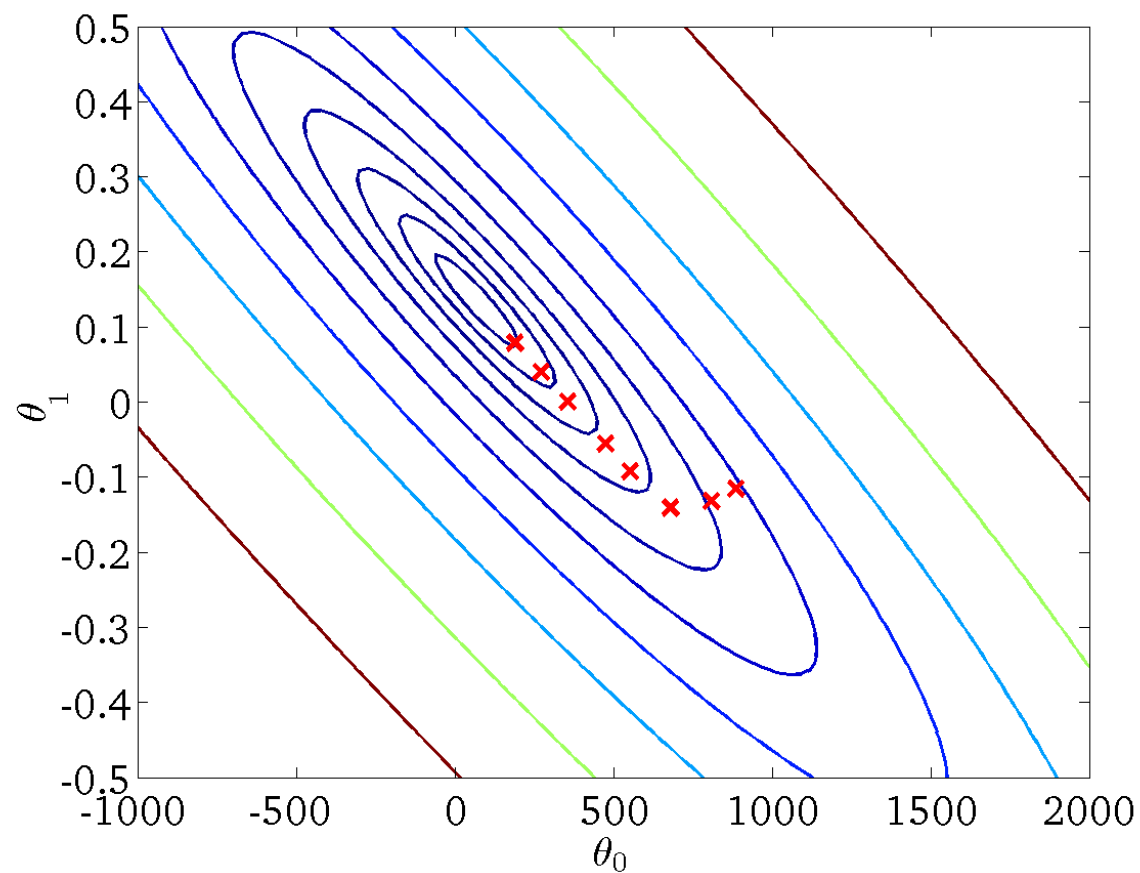
Regressor $f(\theta_0, \theta_1)$

(for fixed θ_0, θ_1 this is a function of x)



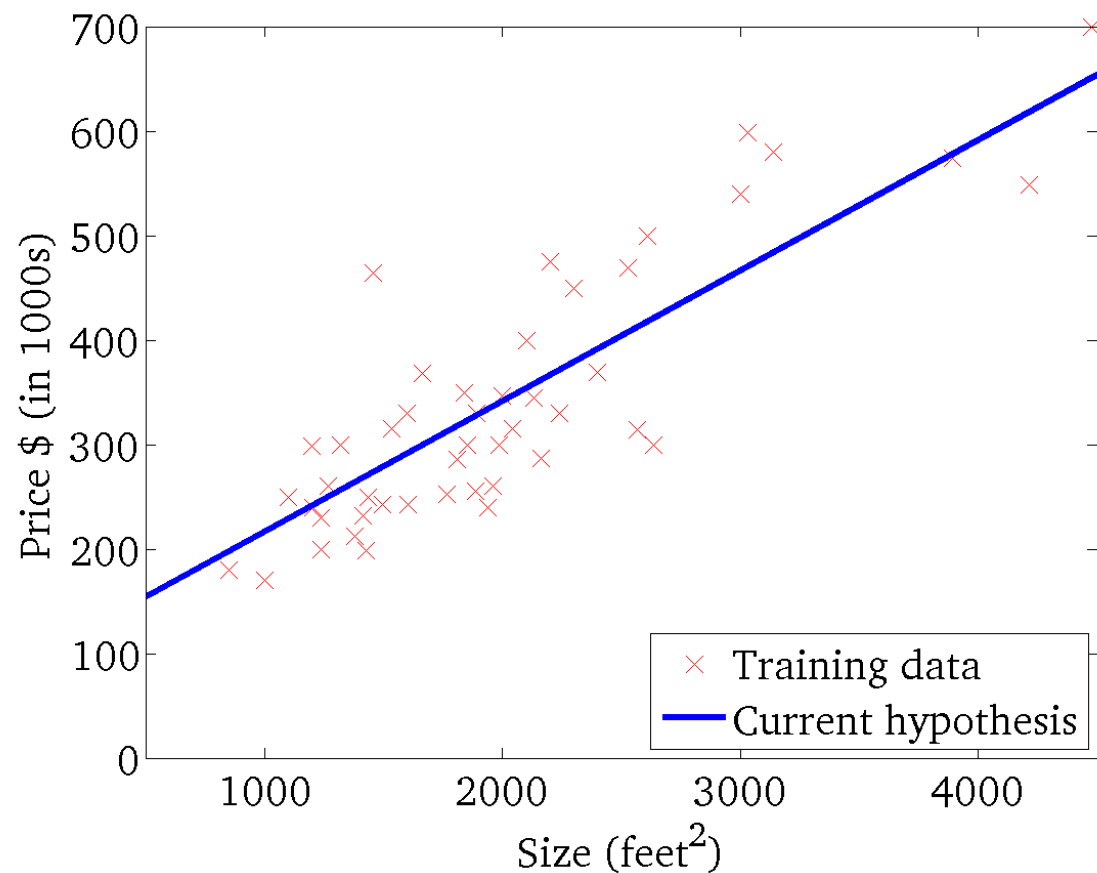
Loss $L(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



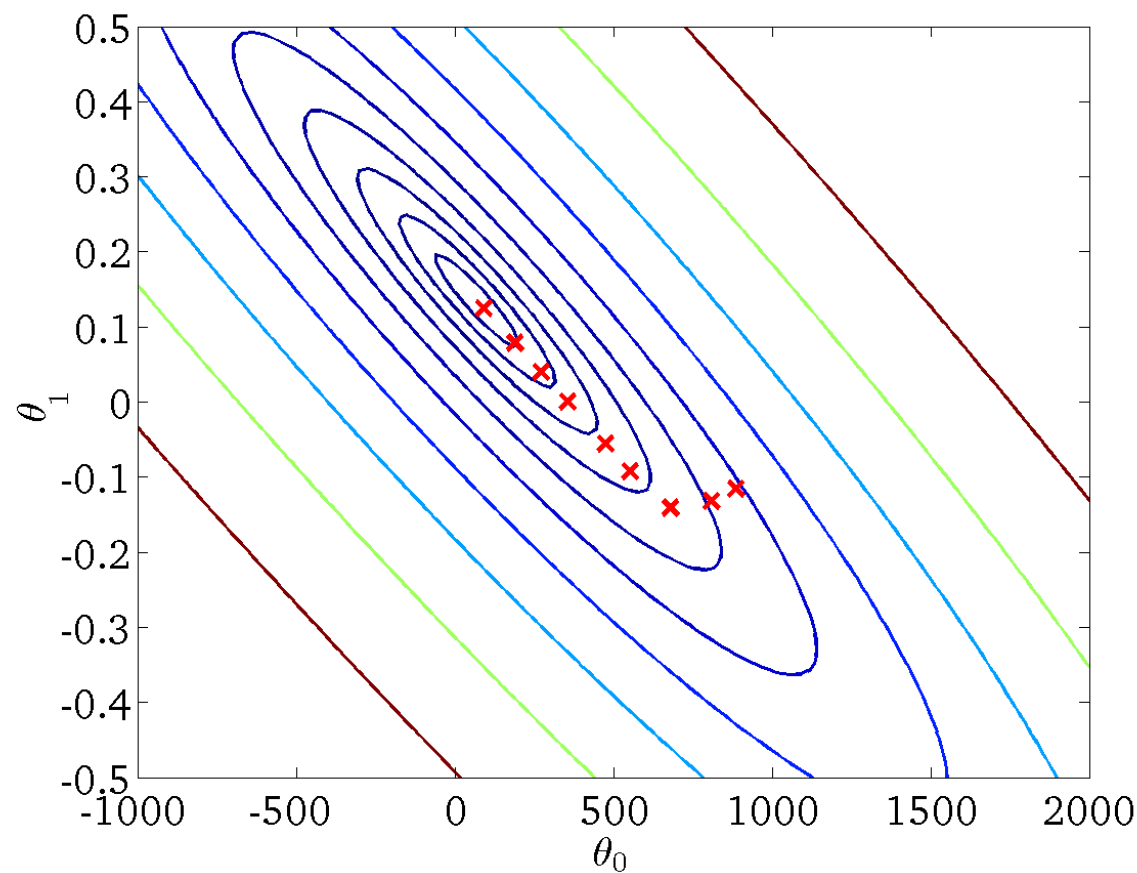
Regressor $f(\theta_0, \theta_1)$

(for fixed θ_0, θ_1 this is a function of x)



Loss $L(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

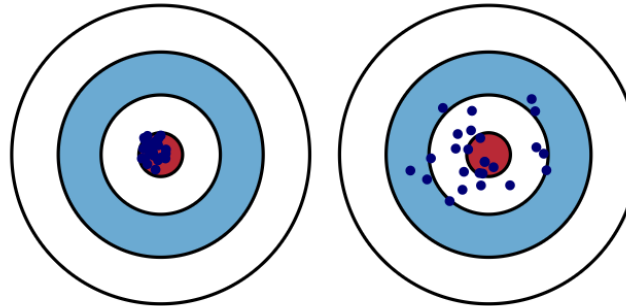


Bias and Variance

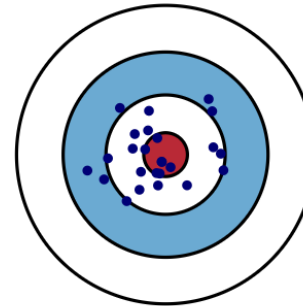
Bias and Variance

- **Bias:** the difference between the average **prediction of model** and the **correct value** which we are trying to predict.
- **Variance:** Model with high variance pays a lot of attention to **training data** and does not generalize on the **test data**.

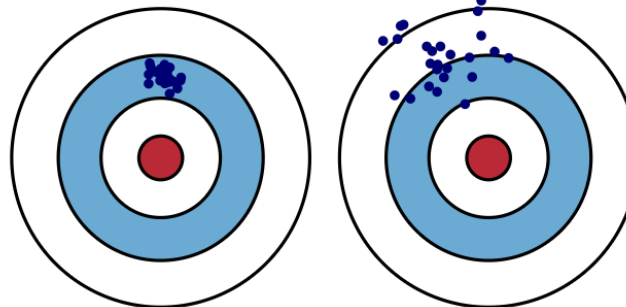
Low bias and
low variance



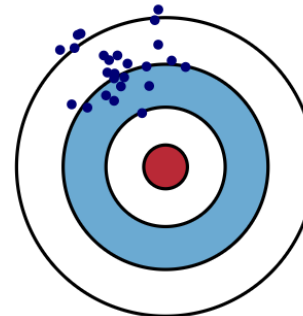
Low bias and
large variance



Large bias and
low variance



Large bias and
large variance



Bias-Variance Tradeoff

- Assume that we have minimized the error (*i.e.*, loss) with respect to **training data**
 - Low **training** error does not imply good expected performance on testing data: **overfitting**
- We would like to reason about the **expected loss** over:
Training Data: $\{(x_1, y_1), \dots, (x_n, y_n)\}$
 - Test point: (x_*, y_*)
- We will decompose the expected loss into:

$$\mathbf{E}_{D, (y_*, x_*)} [(y_* - f(x_*|D))^2] = \text{Noise} + \text{Bias}^2 + \text{Variance}$$

- Define (**unobserved**) the **true** model (h):

$$y_* = h(x_*) + \epsilon_*$$

Assume 0 mean noise
[bias goes in $h(x_*)$]

- Completed the squares with: $h(x_*) = h_*$

$$\mathbf{E}_{D, (y_*, x_*)} [(y_* - f(x_*|D))^2] \quad \text{Expected Loss}$$

$$= \mathbf{E}_{D, (y_*, x_*)} \left[\underbrace{(y_* - h(x_*))}_a + \underbrace{h(x_*) - f(x_*|D)}_b \right]^2$$

$$(a + b)^2 = a^2 + b^2 + 2ab$$

$$= \mathbf{E}_{\epsilon_*} [(y_* - h(x_*))^2] + \mathbf{E}_D [(h(x_*) - f(x_*|D))^2] \\ + 2\mathbf{E}_{D, (y_*, x_*)} [y_* h_* - y_* f_* - h_* h_* + h_* f_*]$$

- Define (unobserved) the true model (h):

$$y_* = h(x_*) + \epsilon_*$$

- Completed the squares with: $h(x_*) = h_*$

$$\mathbf{E}_{D, (y_*, x_*)} [(y_* - f(x_*|D))^2] \text{ Expected Loss}$$

$$= \mathbf{E}_{D, (y_*, x_*)} [(y_* - h(x_*) + h(x_*) - f(x_*|D))^2]$$

$$= \mathbf{E}_{\epsilon_*} [(y_* - h(x_*))^2] + \mathbf{E}_D [(h(x_*) - f(x_*|D))^2]$$

$$+ 2\mathbf{E}_{D, (y_*, x_*)} [y_* h_* - y_* f_* - h_* h_* + h_* f_*]$$

Substitute defn. $y_* = h_* + \epsilon_*$

$$\mathbf{E} [(h_* + \epsilon_*)h_* - (h_* + \epsilon_*)f_* - h_* h_* + h_* f_*] =$$

$$\cancel{h_* h_*} + \mathbf{E}[\epsilon_*] h_* - \cancel{h_*} \mathbf{E}[f_*] - \mathbf{E}[\epsilon_*] f_* - \cancel{h_* h_*} + \cancel{h_*} \mathbf{E}[f_*]$$

- Define (unobserved) the true model (h):

$$y_* = h(x_*) + \epsilon_*$$

- Completed the squares with: $h(x_*) = h_*$

$$\begin{aligned} \mathbf{E}_{D, (y_*, x_*)} [(y_* - f(x_*|D))^2] & \quad \text{Expected Loss} \\ &= \mathbf{E}_{D, (y_*, x_*)} [(y_* - h(x_*) + h(x_*) - f(x_*|D))^2] \\ &= \underbrace{\mathbf{E}_{\epsilon_*} [(y_* - h(x_*))^2]}_{\substack{\text{Noise Term} \\ \text{(out of our control)} \\ \text{☹}}} + \underbrace{\mathbf{E}_D [(h(x_*) - f(x_*|D))^2]}_{\substack{\text{Model Estimation Error} \\ \text{(we want to minimize this)} \\ \text{Expand}}} \end{aligned}$$

- Minimum error is governed by the noise.

- Expanding on the model estimation error:

$$\mathbf{E}_D [(h(x_*) - f(x_*|D))^2]$$

- Completing the squares with $\mathbf{E} [f(x_*|D)] = \bar{f}_*$

$$\mathbf{E}_D [(h(x_*) - f(x_*|D))^2]$$

$$= \mathbf{E} [(h(x_*) - \mathbf{E} [f(x_*|D)] + \mathbf{E} [f(x_*|D)] - f(x_*|D))^2]$$

$$= \mathbf{E} [(h(x_*) - \mathbf{E} [f(x_*|D)])^2] + \mathbf{E} [(f(x_*|D) - \mathbf{E} [f(x_*|D)])^2]$$

$$+ 2\mathbf{E} [h_* \bar{f}_* - h_* f_* - \bar{f}_* f_* + \bar{f}_*^2]$$



$$= h_* \bar{f}_* - h_* \mathbf{E} [f_*] - \bar{f}_* \mathbf{E} [f_*] + \bar{f}_*^2 =$$

$$h_* \bar{f}_* - h_* \bar{f}_* - \bar{f}_* \bar{f}_* + \bar{f}_*^2 = 0$$

- Expanding on the model estimation error:

$$\mathbf{E}_D [(h(x_*) - f(x_*|D))^2]$$

- Completing the squares with $\mathbf{E} [f(x_*|D)] = \bar{f}_*$

$$\begin{aligned} \mathbf{E}_D [(h(x_*) - f(x_*|D))^2] \\ = \underbrace{(h(x_*) - \mathbf{E} [f(x_*|D)])^2}_{(\text{Bias})^2} + \underbrace{\mathbf{E} [(f(x_*|D) - \mathbf{E} [f(x_*|D)])^2]}_{\text{Variance}} \end{aligned}$$

- Tradeoff between bias and variance:
 - **Simple Models:** High Bias, Low Variance $\rightarrow$ Underfitting
 - **Complex Models:** Low Bias, High Variance $\rightarrow$ Overfitting

Bias-Variance Plot

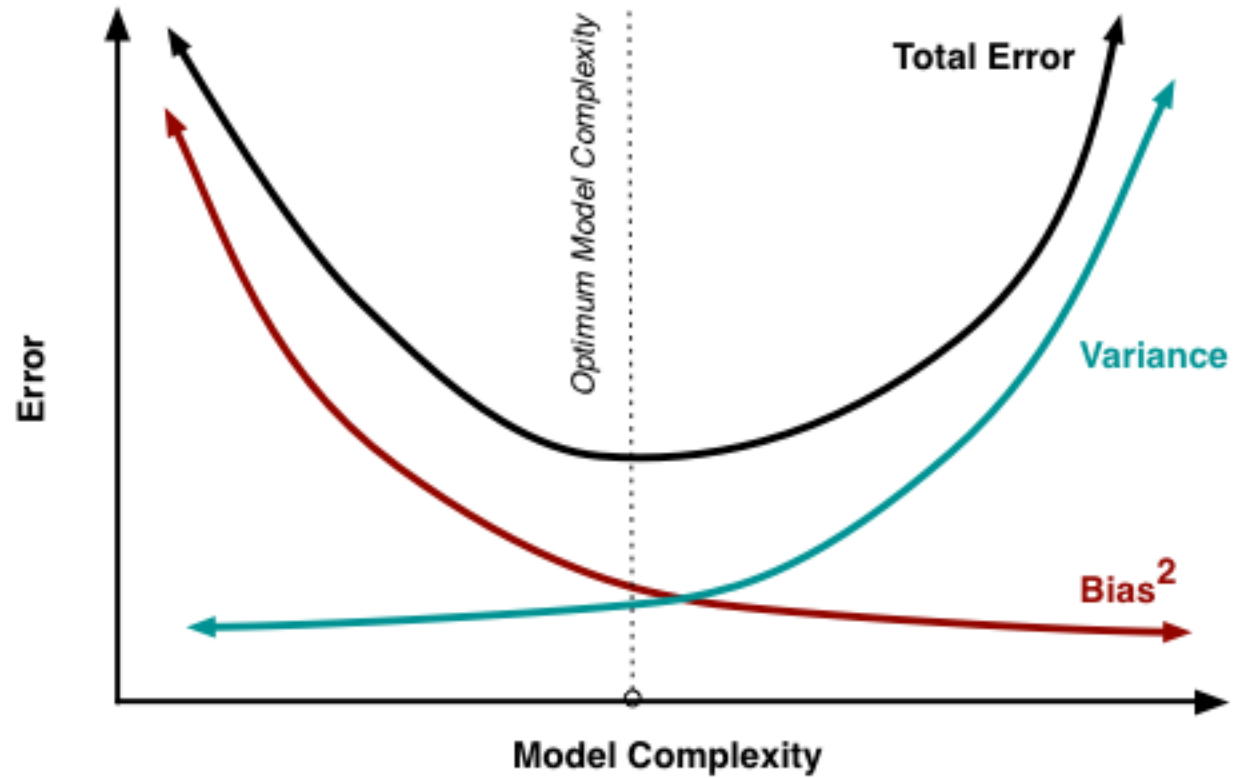
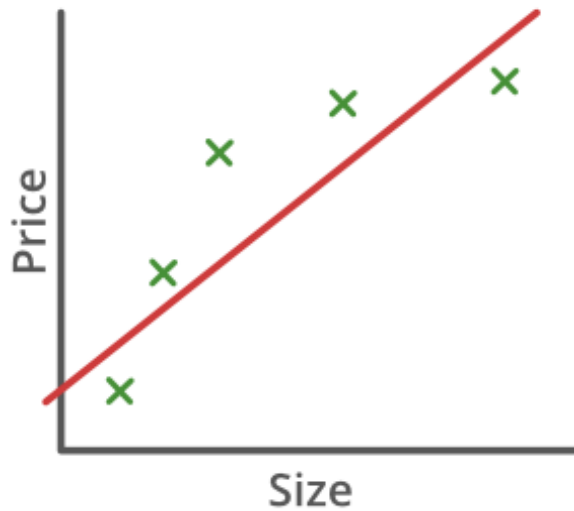


Image from <http://scott.fortmann-roe.com/docs/BiasVariance.html>

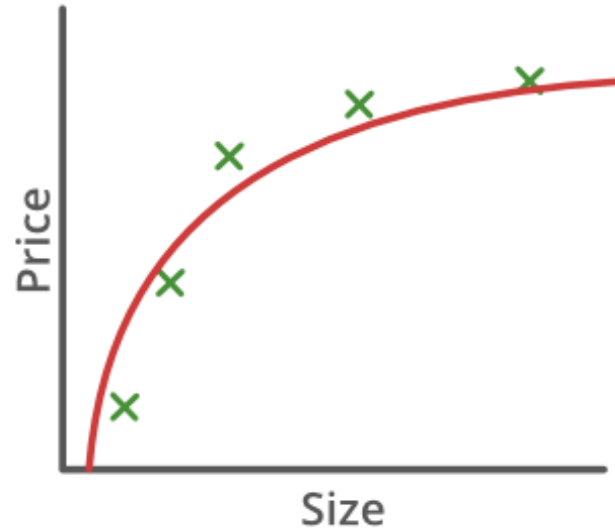
Question

- Which regressor do you think is the best?



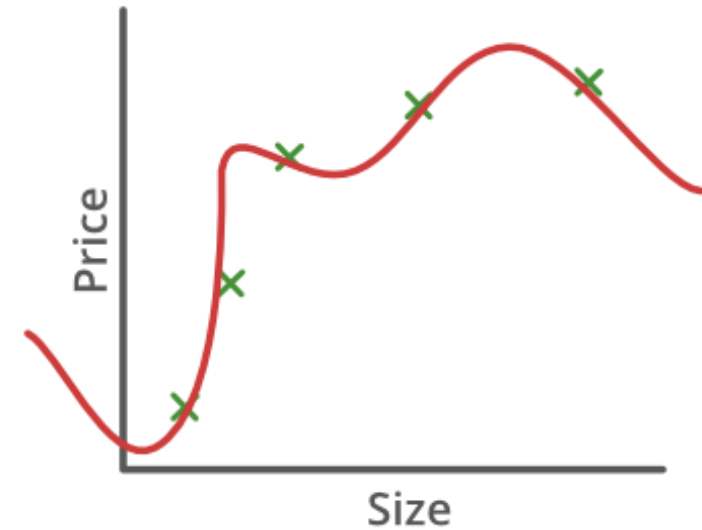
$$\theta_0 + \theta_1 x$$

Regressor 1



$$\theta_0 + \theta_1 x + \theta_2 x^2$$

Regressor 2

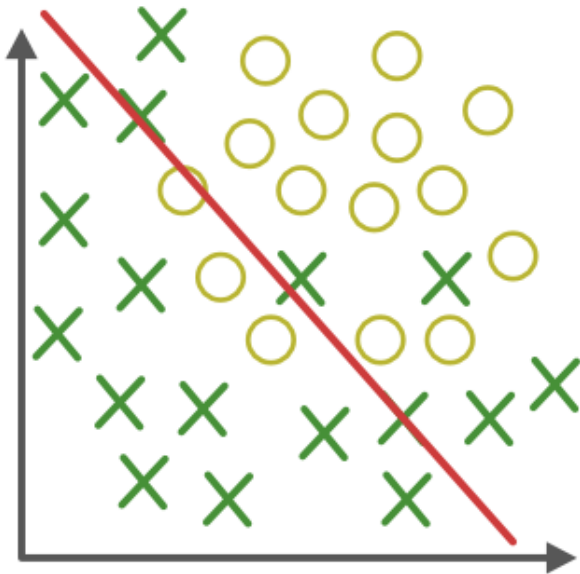


$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

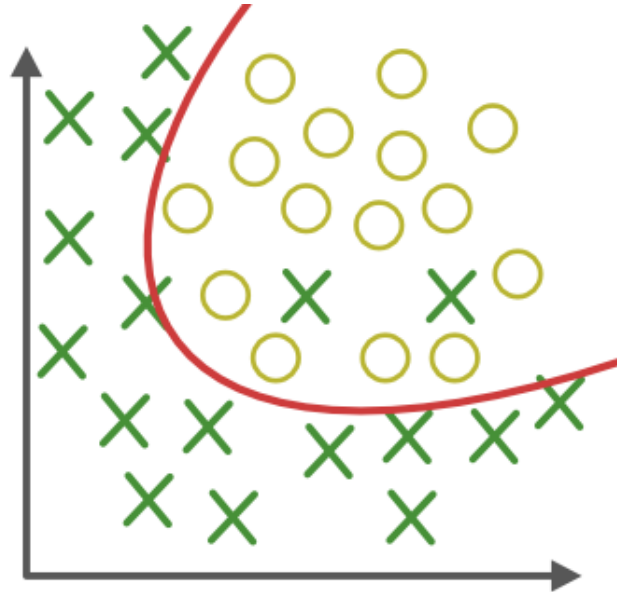
Regressor 3

Question

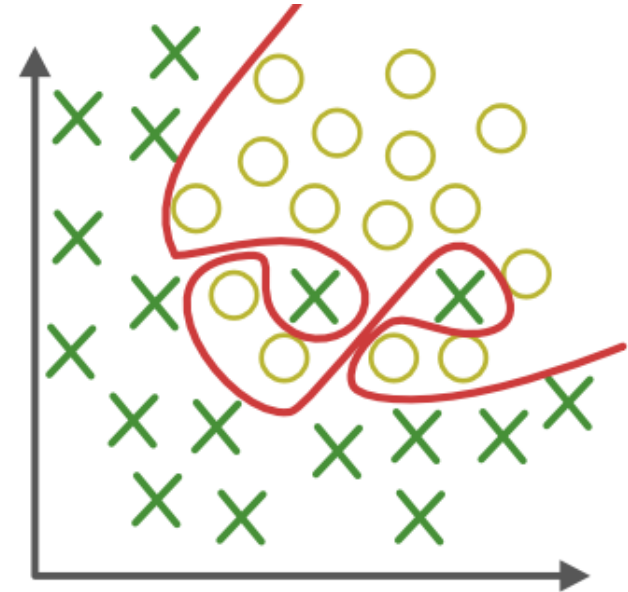
- Which classifier do you think is the best?



Classifier 1

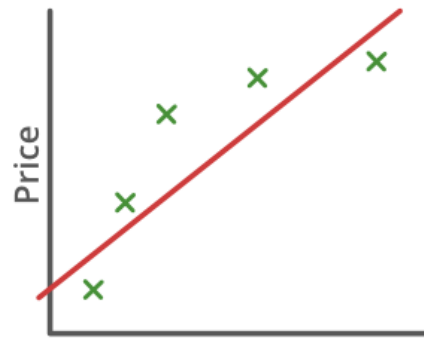


Classifier 2

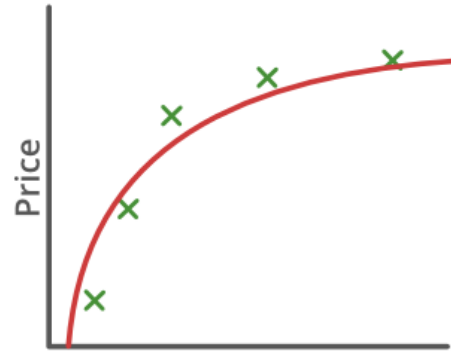


Classifier 3

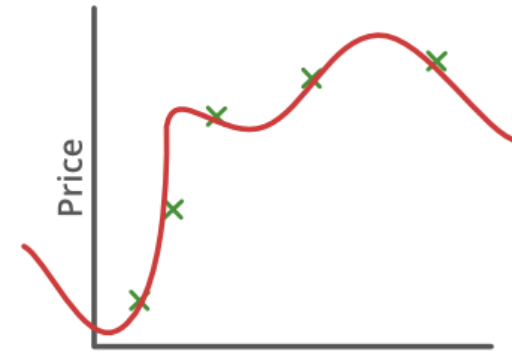
Overfitting and underfitting



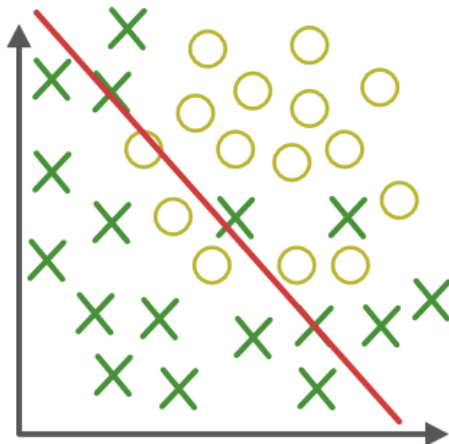
Size
 $\theta_0 + \theta_1 x$



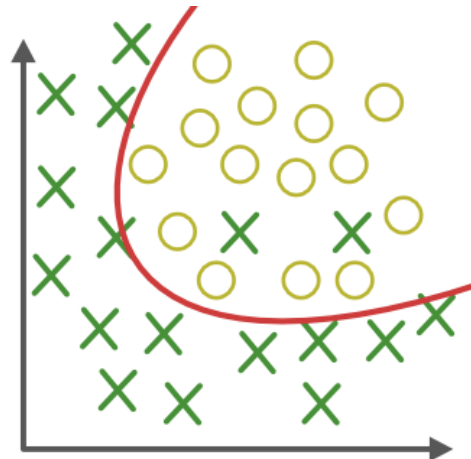
Size
 $\theta_0 + \theta_1 x + \theta_2 x^2$



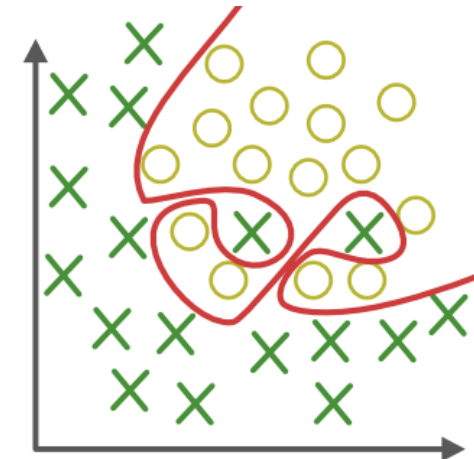
Size
 $\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$



Under-fitting
(too simple to
explain the variance)



Appropriate-fitting



Over-fitting
(forcefitting--too
good to be true) $\mathfrak{G}$

Underfitting

- It cannot capture the underlying trend of the data.
- It destroys the accuracy of our machine learning model.
- **It usually happens when we have less data to build an accurate model and also when we try to build a linear model for a non-linear data.**

Underfitting – High bias and low variance

Techniques to reduce underfitting :

1. Use more data;
2. Increase model complexity;
3. Increase number of features, performing feature engineering;
4. Remove noise from the data;
5. Increase #epochs or increase the duration of training.

Overfitting

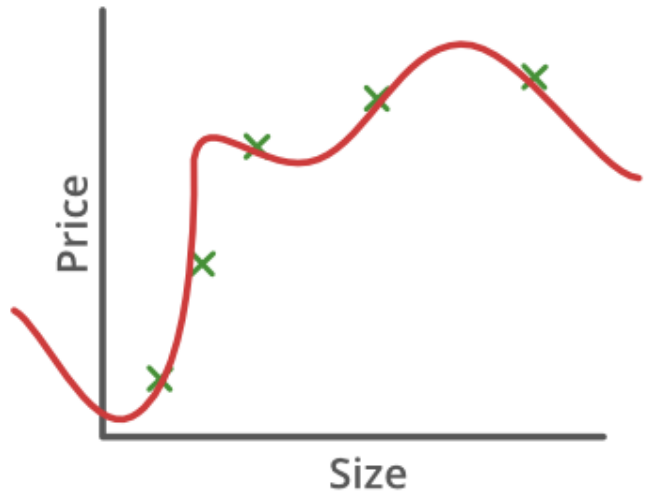
- Limited training data
- Trains the model with **a lot of data** (when a model gets trained with so much of data, it starts learning from the noise and inaccurate data in the dataset).
- High model complexity.

Overfitting – High variance and low bias

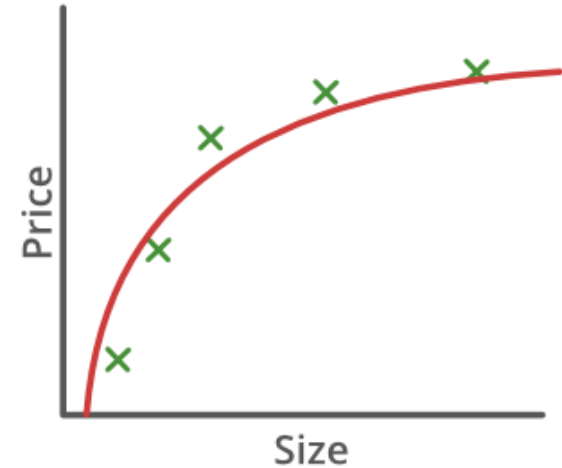
Techniques to reduce overfitting:

1. Increase training data; training data purification;
2. Reduce model complexity;
3. Early stopping during the training phase (have an eye over the loss over the training; as soon as loss begins to increase, stop training);
4. Use dropout for DNNs.

Intuition of Regularization



If we constrain θ_3 and θ_4 to small values



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \cancel{\theta_3 x^3} + \cancel{\theta_4 x^4}$$

$$\approx \theta_0 + \theta_1 x + \theta_2 x^2$$

Regularization

- A **regularizer** is an additional criterion to the loss function to make sure that we don't **overfit**.
- It's called a regularizer since it tries to keep the parameters more normal/regular.
- It is a bias on the model that forces the learning to prefer certain types of weights over others.

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \text{loss}(yy') + \lambda \text{ regularizer}(w, b)$$

Common regularizers

How do we encourage small weights? or penalize large weights?

LASSO Regressor: penalize sum of the weight absolute values (L1 Norm)

$$\arg \min_{\mathbf{w}} L(\mathbf{w}) + \lambda \sum_i |w_i|$$

Ridge Regressor: penalize sum of the squared weights (L2 Norm)

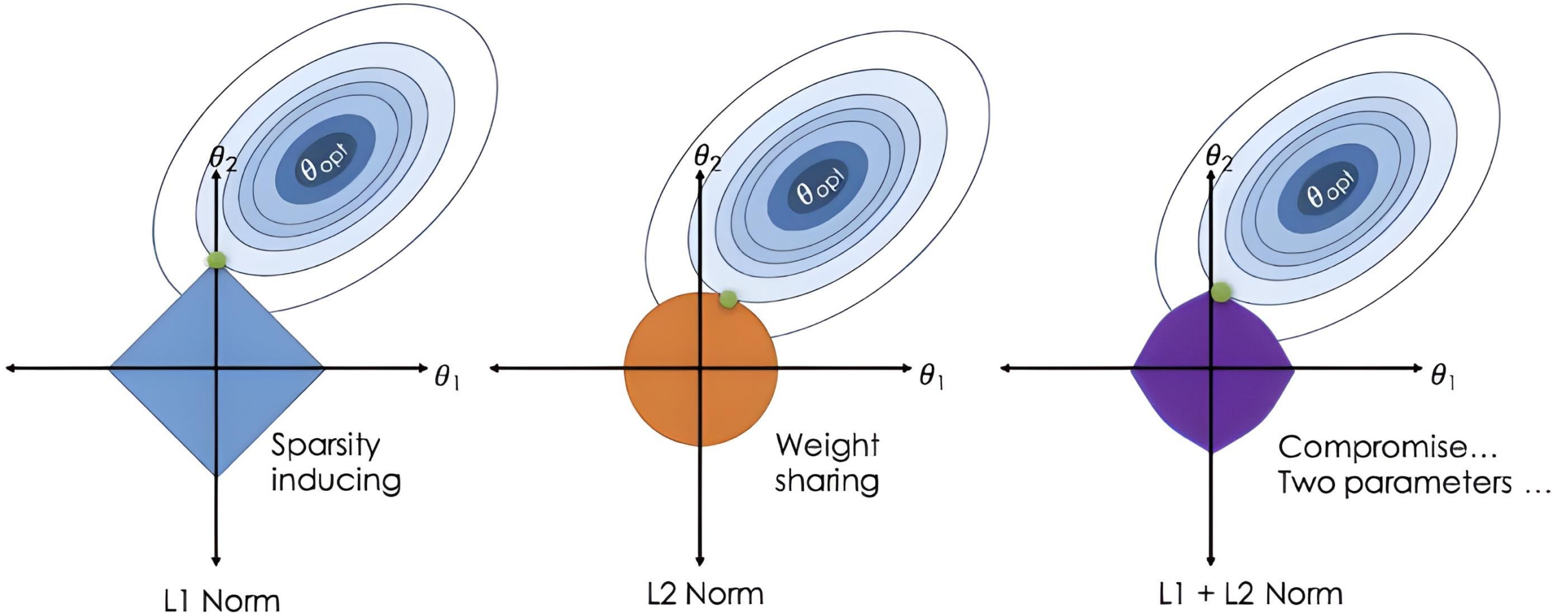
$$\arg \min_{\mathbf{w}} L(\mathbf{w}) + \lambda \sum_i |w_i|^2$$

λ : The [hyperparameter](#) to tune manually

LASSO lead to sparsity

L1 enforce sparse solutions:

The loss contours first hit the sharp corners of the L1 regularizer



Evaluation Metrics and Tuning

Confusion Matrix

		Predicted Class	
		Yes	No
Actual Class	Yes	TP	FN
	No	FP	TN

$$\text{Accuracy} = \frac{TP+TN}{n}$$

- TP: True Positive
- FP: False Positive
- FN: False Negative
- TN: True Negative

Classification Metrics

		Predicted Class	
		Yes	No
Actual Class	Yes	TP	FN
	No	FP	TN

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Accuracy} = \frac{TP + TN}{n}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

- Fraction of actual positives are predicted positive
- Fraction of predicted positives are actual positives

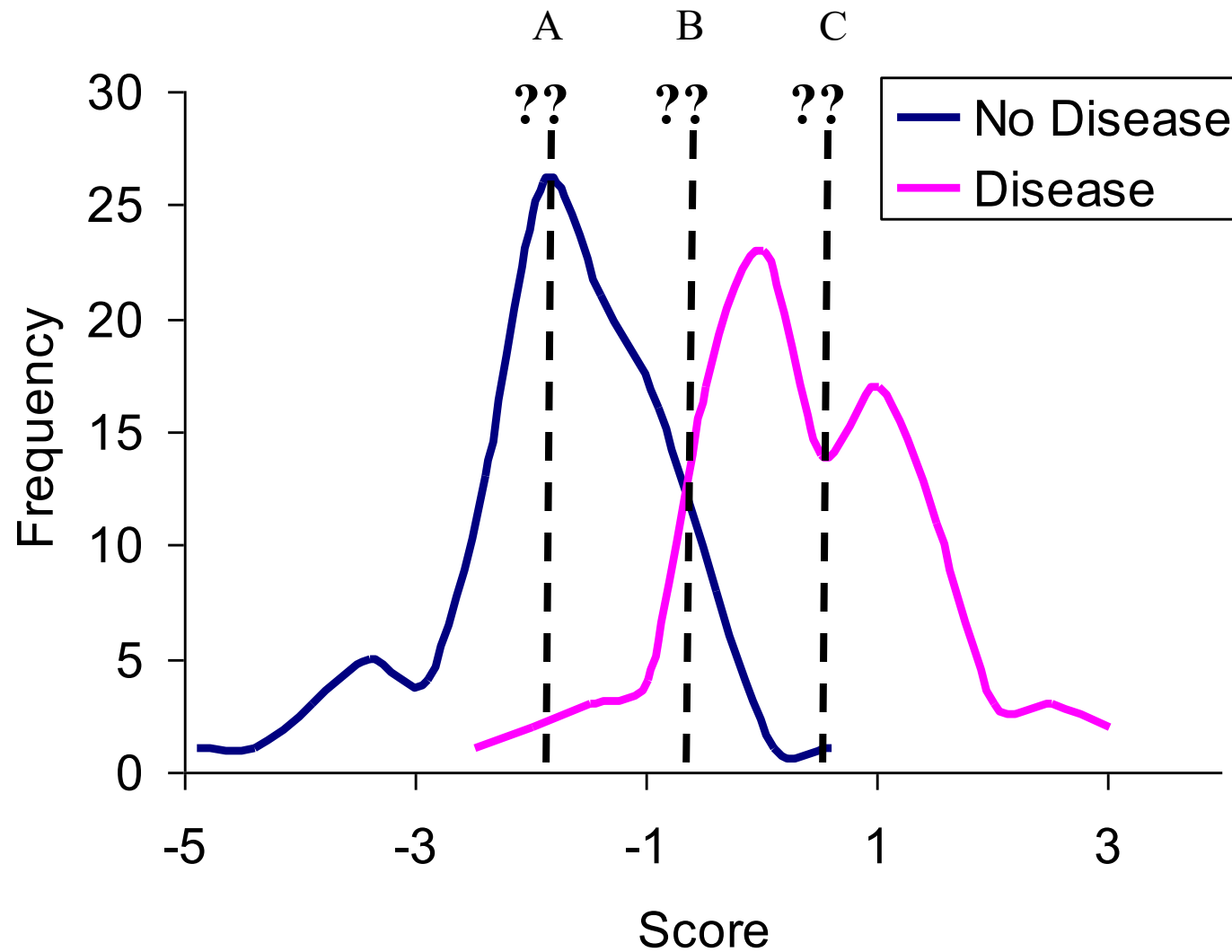
Classification Metrics

F1 score combines both precision and recall

$$\text{F1 Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

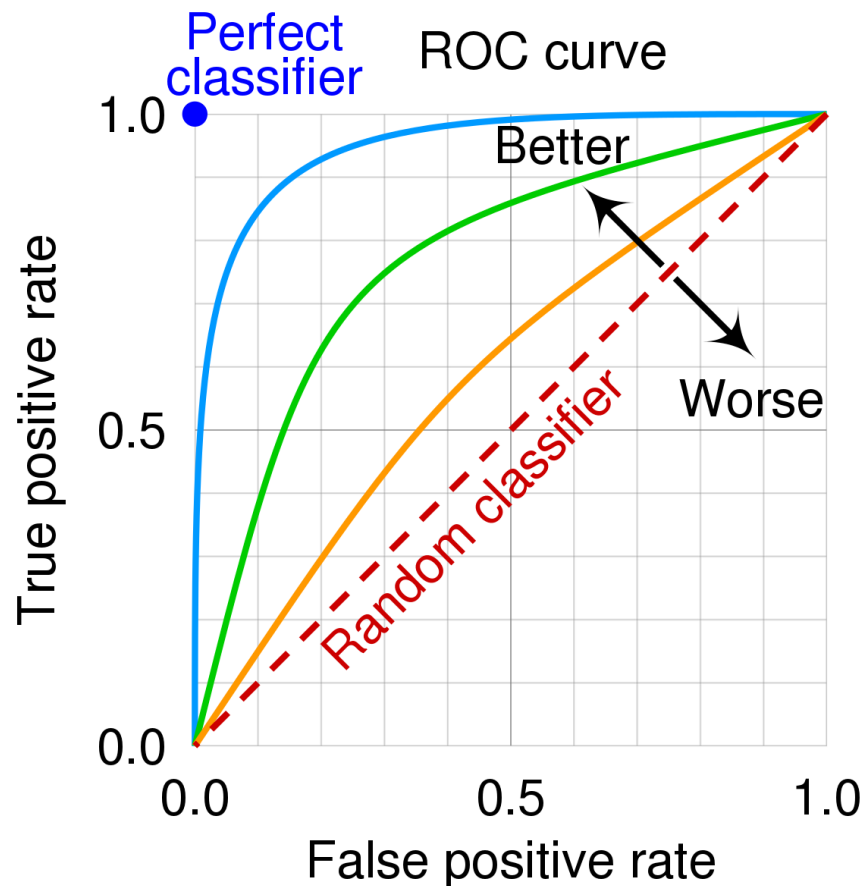
Visual Perspective

The threshold of positive class can be tuned



ROC Curve

Tune the threshold of positive class can obtain the following ROC curve



$$\text{True Positive Rate} = \frac{TP}{TP + FN}$$

$$\text{False Positive Rate} = \frac{FP}{FP + TN}$$

ROC-AUC Score: Area Under the ROC curve

Example

Problem and Data (Breast cancer)

Diagnosis Result	Surgical pathology results		Total
	Patient (P)	Non-patient (N)	
Positive	80	100	180
Negative	20	800	820
Total	100	900	1000

Compute the True Positive Rate and False Positive Rate.

Summary

- If the **True Positive Rate** of a diagnostic test is relatively low, there will be many false-negative patients. This will delay the patient's visit, affect the development and recovery of the disease, and even lead to premature death of the patient.
- If the **False Positive Rate** of a diagnostic test is relatively high, there will be many false positive patients. This will waste medical resources and cause unprovoked panic and anxiety among patients.

Hyper-parameter Tuning and Model Selection

Given Data:

Train data Z_{train}

Test data Z_{test}

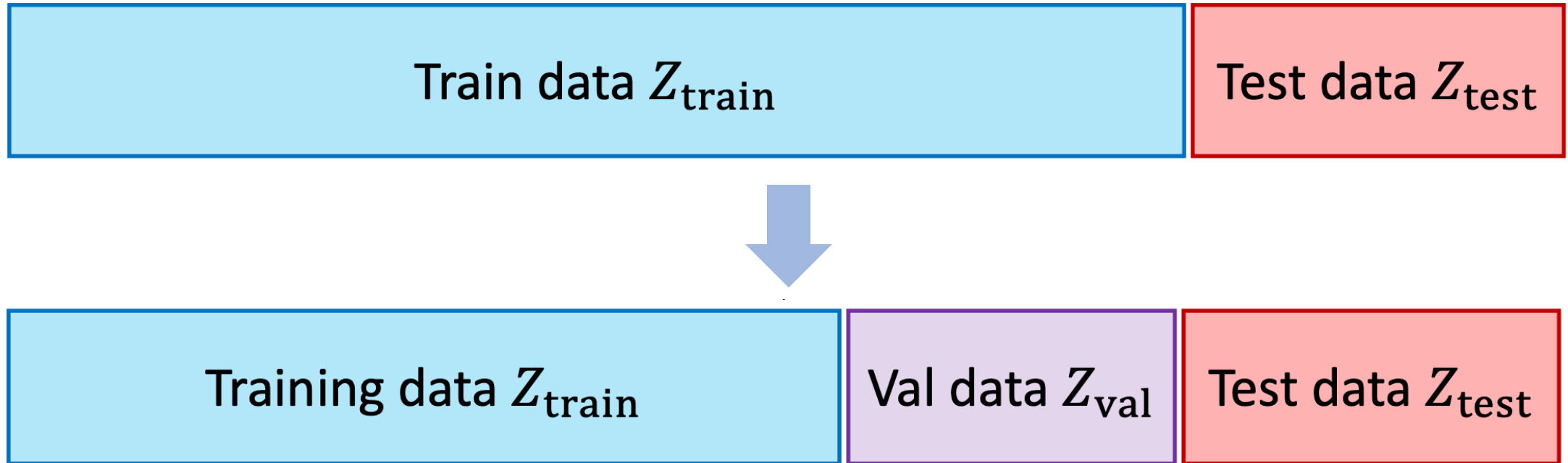
Goal: Choose hyperparameter/model for best generalization ability.

Can we just select hyperparameter that minimizes the test set Z_{test} ?

NO!!! This will lead to the overfitting on the test set.

- This will leakage information of test data
- Results on test data will not accurately reflect the model's generalization ability

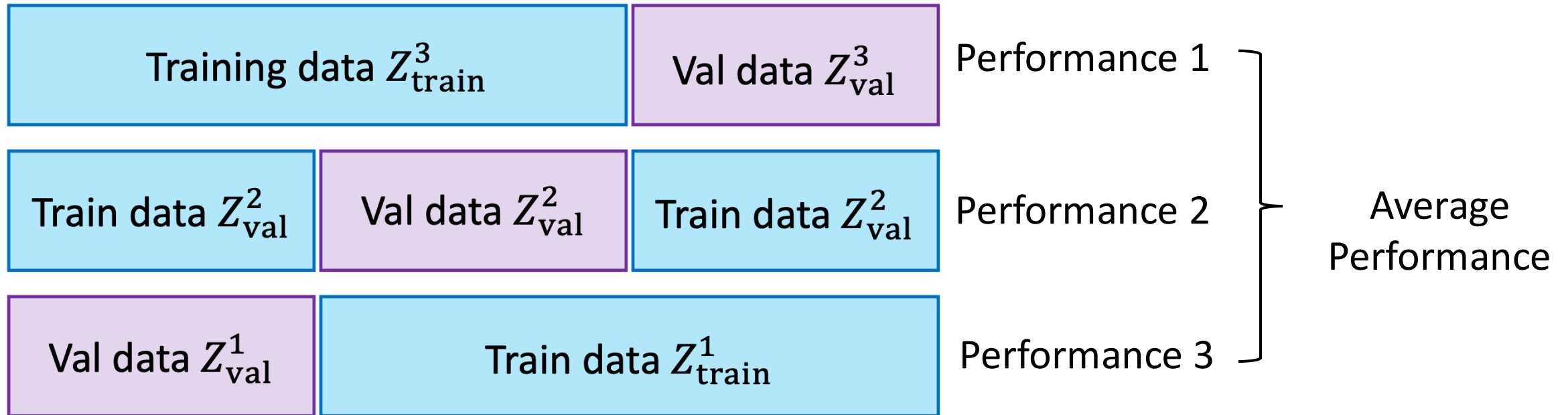
Training/Validation/Test Split



Solution: Tune hyperparameters/models on a **held-out validation data**

K-fold Cross Validation

An example of 3-fold



Tune model based on average performance
Avoid Overfitting on a selected validation set

An Example

Suppose we have a data sample with 6 observations:

1 [0.1, 0.2, 0.3, 0.4, 0.5, 0.6]

- The first step is to pick a value for k in order to determine the number of folds used to split the data.
- Here, we will use a value of $k = 3$. That means we will shuffle the data and then split the data into 3 groups.
- Because we have 6 observations, each group will have an equal number of 2 observations.

An Example

For example:

- 1 Fold1: [0.5, 0.2]
- 2 Fold2: [0.1, 0.3]
- 3 Fold3: [0.4, 0.6]

Three models are trained and evaluated with each fold given a chance to be the held out test set.

For example:

- **Model 1:** Trained on Fold1 + Fold2, Tested on Fold3
- **Model 2:** Trained on Fold2 + Fold3, Tested on Fold1
- **Model 3:** Trained on Fold1 + Fold3, Tested on Fold2

Taking the average of these three models as well as the *std* as the final result.